

11.2 模块开发——数据采集

11.2.1 模拟数据来源

本项目主要通过编写 Java 程序模拟 Nginx 服务生成日志数据，并将生成的日志数据写入到指定的日志文件中供 Flume 采集，具体操作步骤如下。

1. 创建 Maven 项目

在 IDEA 创建一个名为 LogDataSource 的 Maven 项目，并且在项目中创建包 cn.itcast.generate，以及在该包中创建类 GenerateLog，该类用于编写模拟 Nginx 服务生成日志数据的 Java 程序，Maven 项目创建完成的效果如图 11-2 所示。

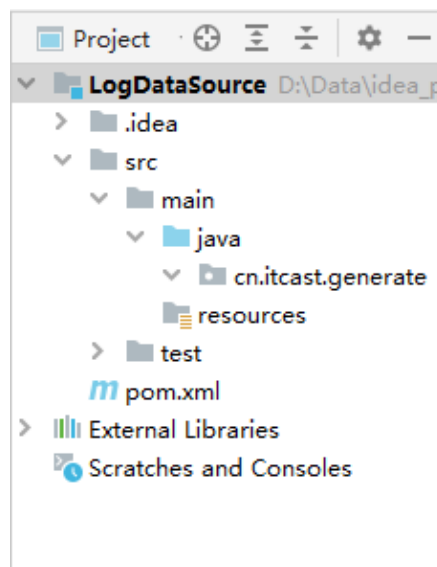


图11-2 创建 Maven 项目 LogDataSource

2. 实现 Java 程序

在包 cn.itcast.generate 下创建类 GenerateLog，在该类中编写模拟 Nginx 服务生成日志数据的代码，具体代码如文件 11-1 所示。

文件11-1 GenerateLog.java

```
1 import java.io.File;
2 import java.io.FileOutputStream;
3 import java.io.IOException;
4 import java.text.DateFormat;
5 import java.text.ParseException;
6 import java.text.SimpleDateFormat;
7 import java.util.Date;
```

```
8 import java.util.Locale;
9 import java.util.Random;
10 import java.util.concurrent.TimeUnit;
11 public class GenerateLog {
12     public static String[] url_paths = {
13         "article/112.html",
14         "article/113.html",
15         "article/114.html",
16         "article/115.html",
17         "article/116.html",
18         "article/117.html",
19         "article/118.html",
20         "article/119.html",
21         "video/821",
22         "tag/list"
23     };
24     public static String[] ip_splices =
25         {"102","71","145","33","67","54","164","121"};
26     public static String[] status_codes = {"200","404","500"};
27     public static void main(String[] args)
28         throws InterruptedException, IOException, ParseException {
29         //运行 jar 文件时, 通过参数指定日志数据的输出目录
30         String path = args[0];
31         File file = new File(path);
32         while (true){
33             FileOutputStream fos = new FileOutputStream(file, true);
34             String content = generateLog()+"\n";
35             fos.write(content.getBytes());
36             //间隔 1 秒钟生成 1 条日志数据
37             TimeUnit.SECONDS.sleep(1);
38             fos.close();
39         }
40     }
41     //随机生成 IP 地址
```

```
42     public static String randomIp(){
43         int ipNum;
44         String ip = "";
45         for (int i=0; i<4; i++){
46             ipNum = new Random().nextInt(8);
47             ip += "."+ip_splices[ipNum];
48         }
49         return ip.substring(1);
50     }
51     //随机生成状态码
52     public static String randomCode(){
53         int codeNum = new Random().nextInt(3);
54         return status_codes[codeNum];
55     }
56     //随机生成 URL
57     public static String randomUrl(){
58         int urlNum = new Random().nextInt(10);
59         return url_paths[urlNum];
60     }
61     //随机生成时间
62     public static String getDate() throws ParseException {
63         Random random = new Random();
64         String date = new String();
65         DateFormat dateFormat =
66             new SimpleDateFormat("dd/MMM/yyyy:HH:mm:ss", Locale.ENGLISH);
67         String days = String.valueOf(random.nextInt(30)+1);
68         String hour = String.valueOf(random.nextInt(24));
69         String minute = String.valueOf(random.nextInt(60));
70         String second = String.valueOf(random.nextInt(60));
71         date = days + "/"
72             + "Aug" + "/"
73             + "2022" + ":"
74             + hour + ":"
75             + minute + ":"
```

```
76         + second;
77     Date parseDate = dateFormat.parse(date);
78     String formatDate = dateFormat.format(parseDate);
79     return formatDate;
80 }
```

上述代码所生成单条日志的数据格式如下。

```
54.102.102.164 [02/Aug/2022:14:51:54 +0800] "GET /article/115.html HTTP/1.1" 500
```

上述所展示的日志数据中，54.102.102.164 表示用户浏览网站时的 IP 地址；02/Aug/2022:14:51:54 表示用户浏览网站时的日期和时间；+0800 表示用户的时区；GET 表示请求方式；/article/115.html 表示用户浏览网站的 URL；HTTP/1.1 表示超文本传输协议的版本为 1.1；500 表示网站的响应状态码。

3. 将 Java 程序封装为 jar 文件

在 IDEA 的右侧单击“Maven”，在弹出的 Maven 窗口依次单击“LogDataSource”→“Lifecycle”折叠框，如图 11-3 所示。

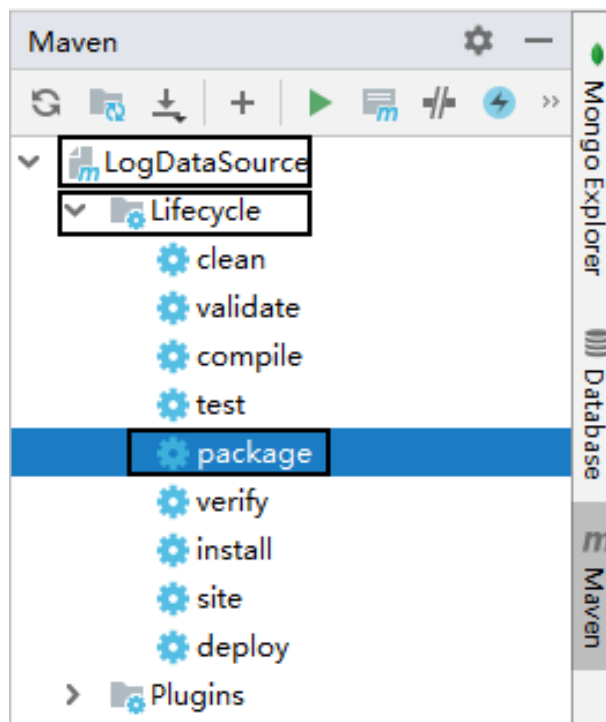
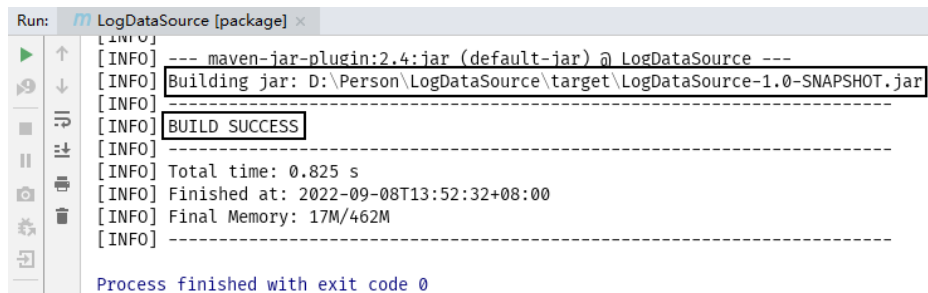


图11-3 打开 Maven 窗口

在图 11-3 中，双击“package”选项将 Maven 项目 LogDataSource 封装为 jar 文件，如图 11-4 所示。



```
Run: LogDataSource [package] x
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ LogDataSource ---
[INFO] Building jar: D:\Person\LogDataSource\target\LogDataSource-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.825 s
[INFO] Finished at: 2022-09-08T13:52:32+08:00
[INFO] Final Memory: 17M/462M
[INFO] -----
Process finished with exit code 0
```

图11-4 封装 jar 文件

在图 11-4 中，若控制台出现“BUILD SUCCESS”的提示信息，说明成功将 Maven 项目 LogDataSource 封装为 jar 文件，此时 jar 文件存放在 D:\Person\LogDataSource\target\目录，并且 jar 文件的名称默认为 LogDataSource-1.0-SNAPSHOT.jar。

4. 上传 jar 文件

在虚拟机 Hadoop1 的 /export/data 目录下执行“rz”命令，上传 jar 文件 LogDataSource-1.0-SNAPSHOT.jar。

5. 创建目录

在虚拟机 Hadoop1 的 /export/data/目录下新建/weblog 目录，用于存放日志文件，具体命令如下。

```
$ mkdir -p /export/data/weblog
```

6. 测试 jar 文件

运行 LogDataSource-1.0-SNAPSHOT.jar 文件，并指定日志数据写入到/export/data/weblog 目录下的 nginx.log 文件中，在虚拟机 Hadoop1 的 /export/data 目录执行如下命令。

```
$ java -cp LogDataSource-1.0-SNAPSHOT.jar cn.itcast.generate.GenerateLog \
/export/data/weblog/nginx.log
```

上述命令执行完成后，LogDataSource-1.0-SNAPSHOT.jar 文件便开始模拟生成日志数据，并且将日志数据写入到/export/data/weblog 目录 nginx.log 文件中。若要停止 jar 文件的运行，则可以通过组合键“Ctrl +C”来实现。

7. 查看日志数据

开启一个新的虚拟机 Hadoop1 窗口，在 /export/data/weblog 目录下执行如下命令查看日志文件 nginx.log 中实时写入的日志数据。

```
$ tail -f /export/data/weblog/nginx.log
```

上述命令执行完成的效果如图 11-5 所示。

IP Address	Timestamp	Offset	Log Entry	Status
164.145.71.145	[29/Aug/2022:22:34:11	+0800]	"GET /article/112.html HTTP/1.1"	500
102.102.54.33	[29/Aug/2022:03:58:29	+0800]	"GET /tag/list HTTP/1.1"	404
54.33.71.54	[25/Aug/2022:17:29:48	+0800]	"GET /article/117.html HTTP/1.1"	404
145.102.71.67	[04/Aug/2022:08:42:37	+0800]	"GET /article/118.html HTTP/1.1"	500
121.71.164.67	[18/Aug/2022:06:09:33	+0800]	"GET /tag/list HTTP/1.1"	200
121.121.67.54	[02/Aug/2022:07:58:07	+0800]	"GET /article/112.html HTTP/1.1"	404
71.33.67.33	[06/Aug/2022:18:52:53	+0800]	"GET /video/821 HTTP/1.1"	404
102.71.102.121	[28/Aug/2022:19:56:18	+0800]	"GET /tag/list HTTP/1.1"	500
121.67.102.145	[23/Aug/2022:09:33:47	+0800]	"GET /article/112.html HTTP/1.1"	200
33.33.33.121	[25/Aug/2022:18:36:11	+0800]	"GET /video/821 HTTP/1.1"	200

图11-5 查看日志数据

从图 11-5 可以看出，日志文件 `nginx.log` 中实时写入了日志数据。若要停止查看日志数据，则可以通过组合键“`Ctrl + C`”来实现。

11.2.2 Flume 采集数据

这里使用 Flume 采集日志文件 `nginx.log` 中的日志数据，并将采集的日志数据写入到 HDFS。在虚拟机 Hadoop1 的 `/export/servers/flume-1.9.0/conf/` 目录下创建文件 `WebLog.conf`，在该文件中指定采集方案，具体内容如文件 11-2 所示。

文件11-2 WebLog.conf

```

1 al.sources = r1
2 al.sinks = k1
3 al.channels = c1
4 al.sources.r1.type = exec
5 al.sources.r1.command = tail -f /export/data/weblog/nginx.log
6 al.sinks.k1.type = hdfs
7 al.sinks.k1.hdfs.path = /weblog/%y-%m-%d/%H-%M/
8 al.sinks.k1.hdfs.filePrefix = itcast-
9 al.sinks.k1.hdfs.fileType = DataStream
10 al.sinks.k1.hdfs.useLocalTimeStamp = true
11 al.channels.c1.type = memory
12 al.channels.c1.capacity = 1000
13 al.channels.c1.transactionCapacity = 100
14 al.sources.r1.channels = c1
15 al.sinks.k1.channel = c1

```

上述指定的采集方案中，指定 Source 类型为 Exec Source，用于持续监控文件 `nginx.log` 内数据的变化，实现实时采集日志数据的功能。指定 Sink 类型为 HDFS Sink，用于将实时采集的日志数据写入 HDFS 的指定目录。指定 Channel 类型为 Memory Channel，用于将实时采集的日志数据缓存在内存中。

```
$ bin/flume-ng agent --name a1 --conf conf/ --conf-file conf/WebLog.conf \
-Dflume.root.logger=INFO,console
```

```
root@hadoop1:/export/servers/flume-1.9.0
```

```
File Edit View Options Transfer Script Tools Window Help
```

```
Enter host -> <Alt>+<N>
```

```
✓ root@hadoop1/export/data ✓ root@hadoop1:/export/servers/flume-1.9.0 ✕ root@hadoop2:~ ✕ root@hadoop3:~
```

```
outerGroup.register(MonitoredCounterGroup.java:119) Monitored counter group for type: SOURCE, name: r1: Successfully registered new MBean.
```

```
2022-09-02 17:21:19.819 [LifecycleSupervisor-1-1] [INFO - org.apache.flume.instrumentation.MonitoredCounterGroup.start(MonitoredCounterGroup.java:95)] Component type[SOURCE, name: r1 started]
```

```
2022-09-02 17:21:19.818 [LifecycleSupervisor-1-2] [INFO - org.apache.flume.instrumentation.MonitoredCounterGroup.register(MonitoredCounterGroup.java:119) Monitored counter group for type: SINK, name: k1: Successfully registered new MBean.
```

```
2022-09-02 17:21:19.819 [LifecycleSupervisor-1-2] [INFO - org.apache.flume.instrumentation.MonitoredCounterGroup.start(MonitoredCounterGroup.java:95)] Component type[SINK, name: k1 started]
```

```
2022-09-02 17:21:23.837 [SinkRunner-PollingRunner-DefaultSinkProcessor] [INFO - org.apache.flume.sink.hdfs.HDFSDataStream.configure(HDFSDataStream.java:57)] Serializer = TEXT, UserRawLocalFileSystem = false
```

```
2022-09-02 17:21:23.933 [SinkRunner-PollingRunner-DefaultSinkProcessor] [INFO - org.apache.flume.sink.hdfs.BucketWriter.open(BucketWriter.java:246)] Creating /weblog/22-09-02/17-21/itcast-.1662110483837.tmp
```

```
2022-09-02 17:21:24.869 [SinkRunner-PollingRunner-DefaultSinkProcessor] [INFO - org.apache.flume.sink.hdfs.BucketWriter.close(BucketWriter.java:438)] Closing /weblog/22-09-02/17-21/itcast-.1662110483837.tmp
```

```
2022-09-02 17:21:24.885 [hdfs-k1-call-runner-3] [INFO - org.apache.flume.sink.hdfs.BucketWriters$7.call(BucketWriter.java:681)] Renaming /weblog/22-09-02/17-21/itcast-.1662110483837.tmp to /weblog/22-09-02/17-21/itcast-.1662110483837
```

```
2022-09-02 17:21:24.902 [SinkRunner-PollingRunner-DefaultSinkProcessor] [INFO - org.apache.flume.sink.hdfs.BucketWriter.open(BucketWriter.java:246)] Creating /weblog/22-09-02/17-21/itcast-.1662110483838.tmp
```

```
D:\log\crtos\version-1.log
```

```
ssh2: ChaCha20-Poly1305: 44 Rows, 101 Cols Xterm CAP NUM
```

从图 11-6 可以看出，Agent 的 Source 和 Sink 已经开始运行，并且将采集的数据写入到 HDFS 指定目录的临时文件中。

Browsing HDFS | 192.168.121.160:9870/explorer.html#/weblog/22-09-02/23-34

Hadoop Overview Datanodes Datanode Volume Failures Snapshot Startup Progress Utilities

Browse Directory

/weblog/22-09-02/23-34

Show 25 entries Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	root	supergroup	800 B	Sep 02 23:34	2	128 MB	itcast-1662132876787
-rw-r--r--	root	supergroup	777 B	Sep 02 23:34	2	128 MB	itcast-1662132876788
-rw-r--r--	root	supergroup	803 B	Sep 02 23:34	2	128 MB	itcast-1662132876789
-rw-r--r--	root	supergroup	559 B	Sep 02 23:35	2	128 MB	itcast-1662132876790

Showing 1 to 4 of 4 entries

Previous 1 Next

从图 11-7 可以看出，/weblog/22-09-02/23-34 目录中存在多个文件，并且这些文件已经由临时文件滚动为目标文件，这些目标文件中存放着采集的日志数据，例如临时文件 itcast-.1662132876787.tmp 已经滚动为目标文件 itcast-.1662132876787。

接下来，我们以查看目标文件 itcast-1662132876787 的前 5 行数据为例，查看采集的数据内容，具体命令如下。

```
$ hdfs dfs -cat /weblog/22-09-02/23-34/itcast-1662132876787 | head -5
```

上述命令执行完成的效果如图 11-8 所示。

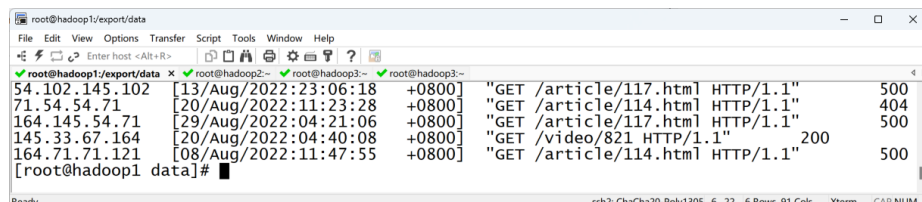


图11-8 查看采集的数据

从图 11-8 可以看出，Flume 采集的数据格式与模拟生成的日志数据格式一致，说明成功从日志文件 nginx.log 中采集模拟生成的日志数据。

11.3 模块开发——数据预处理

从日志文件收集的日志数据，通常情况下，是不能直接进行数据分析的，这是因为日志数据的格式比较复杂，不便于对这些数据进行数据分析的处理，因此对日志数据进行数据分析之前，需要对采集的日志数据进行数据预处理的步骤。

本节我们通过编写 MapReduce 程序对采集的数据进行数据预处理操作，提取每条日志数据中的 IP 地址、用户浏览网站时的日期和时间、请求方式、用户浏览网站的 URL，以及响应状态码，并且对用户浏览网站时的日期和时间进行标准化处理，将默认的日期和时间格式 02/Aug/2022:14:51:54 转换为 2022-08-02 14:51:54。

另外，本次数据预处理的 MapReduce 程序仅仅是对每条日志数据进行提取和转换的处理，并且将每条日志数据处理完的结果直接输出即可，不需要对多条日志数据的处理结果进行聚合操作，因此本节编写的 MapReduce 程序仅需要实现 Map 阶段即可，有关实现数据预处理的步骤如下。

1. 创建 Maven 项目

在 IDEA 创建一个名为 HadoopDataReport 的 Maven 项目，并且在项目中创建包 cn.itcast.hadoop，该包用于存放实现 MapReduce 程序的相关类。

2. 添加项目依赖

在项目 HadoopDataReport 的 pom.xml 文件中添加相关依赖，具体如文件 11-3 所示。

文件11-3 pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```



```
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5 http://maven.apache.org/xsd/maven-4.0.0.xsd">
6      <modelVersion>4.0.0</modelVersion>
7      <groupId>cn.itcast</groupId>
8      <artifactId>HadoopDataReport</artifactId>
9      <version>1.0-SNAPSHOT</version>
10     <properties>
11         <maven.compiler.source>8</maven.compiler.source>
12         <maven.compiler.target>8</maven.compiler.target>
13         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
14     </properties>
15     <dependencies>
16         <dependency>
17             <groupId>org.apache.hadoop</groupId>
18             <artifactId>hadoop-common</artifactId>
19             <version>3.3.0</version>
20         </dependency>
21         <dependency>
22             <groupId>org.apache.hadoop</groupId>
23             <artifactId>hadoop-hdfs</artifactId>
24             <version>3.3.0</version>
25         </dependency>
26         <dependency>
27             <groupId>org.apache.hadoop</groupId>
28             <artifactId>hadoop-client</artifactId>
29             <version>3.3.0</version>
30         </dependency>
31         <dependency>
32             <groupId>org.apache.hadoop</groupId>
33             <artifactId>hadoop-mapreduce-client-core</artifactId>
34             <version>3.3.0</version>
35         </dependency>
36     </dependencies>
37     <build>
```

```
38     <plugins>
39         <plugin>
40             <groupId>org.apache.maven.plugins</groupId>
41             <artifactId>maven-compiler-plugin</artifactId>
42             <version>3.8.1</version>
43             <configuration>
44                 <source>1.8</source>
45                 <target>1.8</target>
46             </configuration>
47         </plugin>
48         <plugin>
49             <groupId>org.apache.maven.plugins</groupId>
50             <artifactId>maven-assembly-plugin</artifactId>
51             <version>3.0.0</version>
52             <configuration>
53                 <descriptorRefs>
54                     <descriptorRef>jar-with-dependencies</descriptorRef>
55                 </descriptorRefs>
56                 <archive>
57                     <manifest>
58                         <mainClass>cn.itcast.hadoop.WebLogDriver</mainClass>
59                     </manifest>
60                 </archive>
61             </configuration>
62             <executions>
63                 <execution>
64                     <phase>package</phase>
65                     <goals>
66                         <goal>single</goal>
67                     </goals>
68                 </execution>
69             </executions>
70         </plugin>
71     </plugins>
```

```
72     </build>
73 </project>
```

上述代码中，第 15~36 行代码用于添加本项目相关的依赖，其中包括 Hadoop 核心依赖、HDFS 依赖、Hadoop 客户端依赖，以及 MapRduce 客户端依赖；第 37~72 行代码用于添加封装 jar 文件的 Maven 插件 maven-compiler-plugin 和 maven-assembly-plugin，前者用于配置 Maven 编译项目的方式，这里指定编译项目的 JDK 版本为 1.8；后者用于开发者在封装 jar 文件时，自定义 jar 文件的内容，这里通过参数 jar-with-dependencies 指定项目封装的 jar 文件包含项目中所有依赖的 jar 文件，以及指定项目封装为 jar 文件之后，运行 jar 文件时默认的程序入口为 cn.itcast.hadoop.WebLogDriver。

3. 实现 Map 阶段

实现 Map 阶段需要用户自定义 Mapper 组件，在 HadoopDataReport 项目的 cn.itcast.hadoop 包中创建用于自定义 Mapper 组件的类 WebLogMap，在该类中编写数据预处理的相关代码，具体如文件 11-4 所示。

文件11-4 WebLogMap.java

```
1  import org.apache.hadoop.io.LongWritable;
2  import org.apache.hadoop.io.NullWritable;
3  import org.apache.hadoop.io.Text;
4  import org.apache.hadoop.mapreduce.Mapper;
5  import java.io.IOException;
6  import java.text.DateFormat;
7  import java.text.ParseException;
8  import java.text.SimpleDateFormat;
9  import java.util.Locale;
10 public class WebLogMap extends Mapper<
11     LongWritable, Text, Text, NullWritable> {
12     @Override
13     protected void map(LongWritable key, Text value, Context context)
14         throws IOException, InterruptedException {
15         DateFormat dateFormatLog =
16             new SimpleDateFormat(
17                 "dd/MMM/yyyy:HH:mm:ss",
18                 Locale.ENGLISH);
19         DateFormat dateFormat =
20             new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
```

```

21      String logData = value.toString();
22      String[] splitLogData = logData.split("\t");
23      String ip = splitLogData[0];
24      String date = "";
25      String request =
26          splitLogData[3].substring(1,splitLogData[3].indexOf(" "));
27      String url = splitLogData[3].substring(
28          splitLogData[3].indexOf(" "),
29          splitLogData[3].lastIndexOf(" ")).trim();
30      String code = splitLogData[4];
31      try {
32          date = dateFormat.format(
33              dateFormatLog.parse(
34                  splitLogData[1].substring(1)));
35      } catch (ParseException e) {
36          throw new RuntimeException(e);
37      }
38      request = ip + "," + date + "," + request + "," + url + "," + code;
39      context.write(new Text(request),NullWritable.get());
40  }
41  }

```

上述代码中，第 23 行代码用于从每条日志数据中提取 IP 地址。第 25~26 行代码用于从每条日志数据中提取请求方式。第 27~29 行代码，用于从每条日志数据中提取用户浏览网站的 URL。第 30 行代码，用于从每条日志数据中提取响应状态码。第 32~34 行代码，用于从每条日志数据中提取用户浏览网站时的日期和时间，并且对日期和时间进行标准化处理，将日期和时间格式化为指定格式。第 38 行代码，用于通过字符“,” 拼接 IP 地址、用户浏览网站时的日期和时间、请求方式、用户浏览网站的 URL、响应状态码。

4. 实现驱动类

在 HadoopDataReport 项目的 cn.itcast.hadoop 包中创建 MapReduce 程序的驱动类 WebLogDriver，在该类中编写配置 MapReduce 程序的相关代码，具体如文件 11-5 所示。

文件11-5 WebLogDriver.java

```

1  import org.apache.hadoop.conf.Configuration;
2  import org.apache.hadoop.fs.Path;
3  import org.apache.hadoop.io.NullWritable;

```

```

4  import org.apache.hadoop.io.Text;
5  import org.apache.hadoop.mapreduce.Job;
6  import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
7  import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
8  import java.io.IOException;
9  import java.text.DateFormat;
10 import java.text.SimpleDateFormat;
11 import java.util.Date;
12 public class WebLogDriver {
13     public static void main(String[] args) throws IOException,
14         InterruptedException, ClassNotFoundException {
15         DateFormat dateFormat =
16             new SimpleDateFormat("yyyy-MM-dd-HH-mm-ss");
17         String date = dateFormat.format(new Date());
18         Configuration configuration = new Configuration();
19         configuration.set("mapreduce.input.fileinputformat.input.dir.recursive", "true");
20         Job job = Job.getInstance(configuration);
21         job.setInputFormatClass(CombineTextInputFormat.class);
22         job.setJarByClass(WebLogDriver.class);
23         job.setMapperClass(WebLogMap.class);
24         job.setMapOutputKeyClass(Text.class);
25         job.setMapOutputValueClass(NullWritable.class);
26         FileInputFormat.setInputPaths(job, new Path("/weblog"));
27         FileOutputFormat.setOutputPath(job, new Path("/output/weblog/"+date));
28         Boolean result = job.waitForCompletion(true);
29         System.exit(result ? 0 : 1);
30     }
31 }

```

上述代码中，第 19 行代码指定参数 `mapreduce.input.fileinputformat.input.dir.recursive` 的参数值为 `true`，用于配置 MapReduce 程序可以递归处理指定目录及子目录中的所有文件。第 21 行代码用于合并小文件，因为 Flume 采集的日志数据上传到 HDFS 之后会产生大量的小文件，如果直接使用 MapReduce 程序对这些小文件进行处理的话，会影响执行效率。第 26 行代码，用于指定文件的所在目录，该目录及其子目录的文件中存放了 Flume 采集的日志数据。第 27 行代码，用于指定 MapReduce 程序处理结果的输出目录，其中 `date` 用于根据

本地系统时间生成目录的名称，目的是为了使 MapReduce 程序将每次的处理结果输出到不同的目录中。

5. 将 MapReduce 程序封装 jar 文件

在 IDEA 的右侧单击“Maven”，在弹出的 Maven 窗口依次单击“HadoopDataReport”→“Lifecycle”，在 Lifecycle 折叠框中双击“package”选项将 Maven 项目 HadoopDataReport 封装为 jar 文件。

jar 文件封装完成后会在 HadoopDataReport 项目所在目录的 target 目录下生成 HadoopDataReport-1.0-SNAPSHOT.jar 和 HadoopDataReport-1.0-SNAPSHOT-jar-with-dependencies.jar，前者是不包含 HadoopDataReport 项目依赖的 jar 文件；后者是包含 HadoopDataReport 项目依赖的 jar 文件，后续将包含 HadoopDataReport 项目依赖的 jar 文件提交到 YARN 集群执行，为了便于后续使用，这里将 jar 文件重命名为 clean.jar。

6. 上传 jar 文件

将 clean.jar 文件上传到虚拟机 Hadoop1 的/export/data 目录。

7. 测试 jar 文件

确保 Hadoop 集群处于启动状态下运行 clean.jar，在虚拟机 Hadoop1 的/export/data 目录执行如下命令。

```
$ hadoop jar clean.jar
```

上述命令执行完成后，会根据 MapReduce 程序在虚拟机 Hadoop1 执行时的系统日期和时间，在 HDFS 的/output/weblog 目录下生成对应的目录，该目录中存放了 MapReduce 程序的处理结果，如图 11-9 所示。

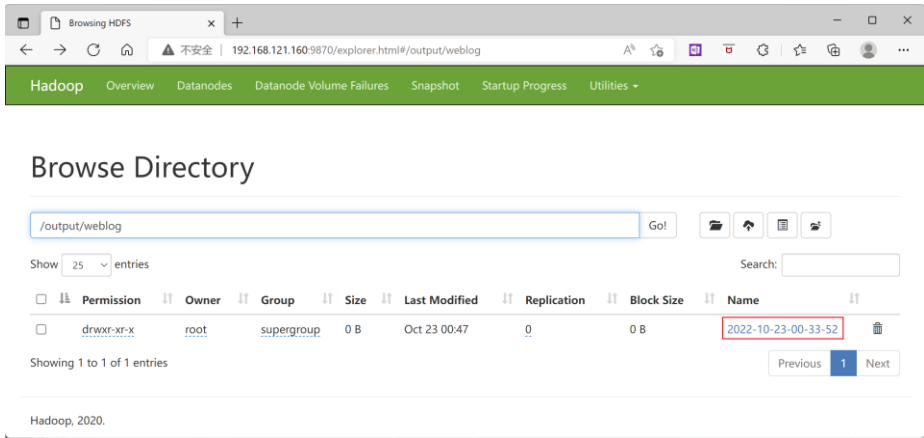


图11-9 MapReduce 程序生成的目录

从图 11-9 可以看出，HDFS 的/output/weblog 目录下生成了一个名为 2022-10-23-00-33-52 的目录。需要注意的是，2022-10-23-00-33-52 目录的名称是根据虚拟机 Hadoop1 的系统时间生成的，读者在进行实际操作时，/output/weblog 目录下生成目录的名称会有所不同。

8. 查看处理结果

在虚拟机 Hadoop1 执行 “hdfs dfs -cat /output/weblog/ 2022-10-23-00-33-52/part-r-00000 | head -5” 命令，查看本次 MapReduce 程序处理结果的前 5 条数据，如图 11-10 所示。

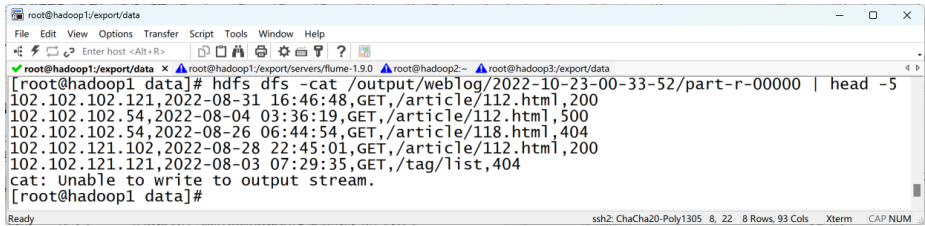


图11-10 查看 MapReduce 程序处理结果的前 5 条数据

从图 11-10 可以看出，MapReduce 程序的处理结果与数据预处理指定的处理逻辑一致，说明 MapReudce 程序成功对采集的日志数据进行数据预处理。

11.4 模块开发——数据仓库开发

数据仓库开发主要是通过 Hive 构建数据仓库，通过数据仓库中的表来存储与需求分析相关的数据，以及分析结果的数据。本节将讲解如何在 Hive 中构建数据仓库。

11.4.1 设计数据仓库

针对此次网站流量日志数据分析系统，需要在 Hive 数据仓库中创建数据仓库层对应的明细表 weblog_dws、中间表 weblog_dwm、业务表 weblog_pv_dwd 和业务表 weblog_avguv_dwd，具体介绍如下。

1. 明细表 weblog_dws

明细表 weblog_dws 存在于数据仓库层的明细层，其做用是存储通过 MapReduce 程序进行数据预处理之后的数据，其表结构如表 11-1 所示。

表11-1 明细表 weblog_dws 的表结构

字 段	数据类型	描 述
visit_ip	STRING	用户访问网站时的 IP 地址
visit_date	STRING	用户访问网站时的日期和时间
visit_request	STRING	用户访问网站时的请求方式
visit_url	STRING	用户访问网站的 URL 地址
visit_code	STRING	用户访问网站的响应状态码

从表 11-1 可以看出，明细表 weblog_dws 包含 5 个字段，这 5 个字段分别用于存储数据预处理之后每条数据中的 IP 地址、请求方式、URL、响应状态码，以及日期和时间。

2. 中间表 weblog_dwm

明细表 weblog_dws 中字段 visit_date 存储的数据包含用户访问网站时的日期和时间，后续不利于通过日期来统计浏览量和人均浏览页，因此这里对明细表 weblog_dws 的数据进行处理，并将处理结果存储在中间表 weblog_dwm，其处理过程是将字段 visit_date 的数据拆分为日期和时间两部分，并分别将这两部分数据存储在字段 visit_split_date 和 visit_split_time。

中间表 weblog_dwm 存在于数据仓库层的中间层，其表结构如表 11-2 所示。

表11-2 中间表 weblog_dwm

字 段	数据类型	描 述
visit_ip	STRING	用户访问网站时的 IP 地址
visit_split_date	STRING	用户访问网站时的日期
visit_split_time	STRING	用户访问网站时的时间
visit_request	STRING	用户访问网站时的请求方式
visit_url	STRING	用户访问网站的 URL 地址
visit_code	STRING	用户访问网站的响应状态码

需要说明的是，中间表 weblog_dwm 是一个分区表，并且将字段 visit_split_date 做为分区字段，其目的是将明细表 weblog_dws 的处理结果插入到中间表 weblog_dwm 时，可以根据用户访问网站时的日期实现动态分区。

3. 业务表 weblog_pv_dwd

业务表 weblog_pv_dwd 存在于数据仓库层的业务层，其做用是存储浏览量分析的结果，其表结构如表 11-3 所示。

表11-3 业务表 weblog_pv_dwd

字 段	数据类型	描 述
datestr	STRING	日期
pv	BIGINT	浏览量

4. 业务表 weblog_avguv_dwd

业务表 weblog_avguv_dwd 存在于数据仓库层的业务层，其做用是存储人均浏览页分析的结果，其表结构如表 11-4 所示。

表11-4 业务表 weblog_avguv_dwd

字 段	数据类型	描 述
datestr	STRING	日期
avguv	INT	人均浏览页面的数量

11.4.2 实现数据仓库

本项目使用远程模式部署的 Hive 实现数据仓库的构建,接下来,我们在虚拟机 Hadoop3 启动 MetaStore 和 HiveServer2 服务,在虚拟机 Hadoop2 使用 Hive 提供的客户端工具 Beeline 连接 HiveServer2 服务实现数据仓库的构建,具体步骤如下。

1. 创建数据库 weblog_dw

在 Hive 中创建数据库 weblog_dw,该数据库用于存储数据仓库层的表,命令如下。

```
create database if not exists weblog_dw;
```

2. 创建明细表 weblog_dws

在数据库 weblog_dw 中创建明细表 weblog_dws,并且向该表中导入 MapReduce 程序对采集的数据进行数据预处理后生成的结果文件,具体内容如下。

(1) 创建明细表 weblog_dws,命令如下。

```
create table if not exists
weblog_dw.weblog_dws (
    visit_ip    string,
    visit_date  string,
    visit_request string,
    visit_url   string,
    visit_code  string
)
row format delimited
fields terminated by ','
lines terminated by '\n';
```

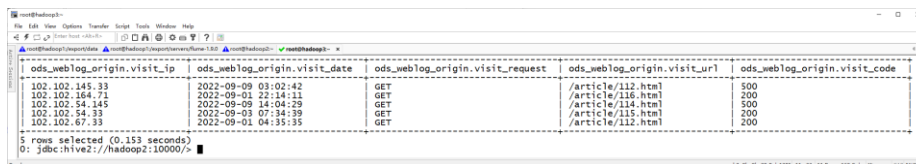
(2) 通过加载文件的方式,将 MapReduce 程序在 HDFS 的目录/output/weblog/2022-10-23-00-33-52 中生成的结果文件 part-r-00000,导入到明细表 weblog_dws,命令如下。

```
load data inpath '/output/weblog/2022-10-23-00-33-52/part-r-00000'
into table weblog_dw.weblog_dws;
```

(3) 查询明细表 weblog_dws 的前 5 条数据,验证 MapReduce 程序结果文件中的数据是否导入到明细表 weblog_dws,命令如下。

```
select * from weblog_dw.weblog_dws limit 5;
```

上述命令的执行效果如图 11-11 所示。



ods_weblog_origin.visit_ip	ods_weblog_origin.visit_date	ods_weblog_origin.visit_request	ods_weblog_origin.visit_url	ods_weblog_origin.visit_code
102.102.145.33	2022-09-09 03:02:42	GET	/article/112.html	300
102.102.164.71	2022-09-01 21:14:11	GET	/article/116.html	200
102.102.54.145	2022-09-09 14:04:39	GET	/article/114.html	500
102.102.54.33	2022-09-03 07:14:39	GET	/article/115.html	200
102.102.67.33	2022-09-01 04:35:35	GET	/article/112.html	200

图11-11 查询明细表 weblog_dws 的前 5 条数据

从图 11-11 可以看出，明细表 weblog_dws 存在数据，并且每行数据与 MapReduce 程序的结果文件中每条数据的内容一致，说明成功将 MapReduce 程序结果文件中的数据导入到明细表 weblog_dws。

3. 创建中间表 weblog_dwm

在数据库 weblog_dw 中创建中间表 weblog_dwm，并且将明细表 weblog_dws 中字段 visit_date 的数据进行拆分处理后的结果插入到中间表 weblog_dwm，具体内容如下。

(1) 创建中间表 weblog_dwm，命令如下。

```
create table if not exists
weblog_dw.weblog_dwm(
    visit_ip      string,
    visit_split_time  string,
    visit_request  string,
    visit_url      string,
    visit_code     string
) partitioned by (visit_split_date string)
row format delimited
fields terminated by ','
lines terminated by '\n';
```

(2) 由于这里以动态分区的方式，将明细表 weblog_dws 的处理结果导入到中间表 weblog_dwm，所以需要开启 Hive 动态分区，命令如下。

```
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nostrict;
```

(3) 通过查询插入的方式，将明细表 weblog_dws 的处理结果插入到中间表 weblog_dwm，命令如下。

```
insert overwrite table weblog_dw.weblog_dwm partition (visit_split_date)
select visit_ip,split(visit_date,' ')[1],visit_request,visit_url,
visit_code,split(visit_date,' ')[0] from weblog_dw.weblog_dws;
```

上述命令中，使用 Hive 内置函数 split()，根据分隔符“ ”对字段 visit_date 的数据进行拆分，其中 split(visit_date,' ')[1]表示获取字段 visit_date 中数据的时间；split(visit_date,' ')[0]表示获取字段 visit_date 中数据的日期。

(4) 查询中间表 weblog_dwm 中分区为 visit_split_date=2022-08-08 的前 5 条数据，验证明细表 weblog_dws 的处理结果是否成功导入到中间表 weblog_dwm，命令如下。

```
select * from weblog_dw.weblog_dwm where visit_split_date='2022-08-08'
```

```
limit 5;
```

上述命令的执行效果如图 11-12 所示。



weblog_dwm.visit_ip	weblog_dwm.visit_split_time	weblog_dwm.visit_request	weblog_dwm.visit_url	weblog_dwm.visit_code	weblog_dwm.visit_split_date
102.102.121.121	23:47:28	GET	/article/114.html	500	2022-08-08
102.102.145.102	04:21:06	GET	/article/113.html	500	2022-08-08
102.102.67.54	20:05:20	GET	/article/116.html	404	2022-08-08
102.102.71.164	08:10:26	GET	/article/114.html	200	2022-08-08
102.121.121.102	03:58:59	GET	/article/117.html	500	2022-08-08

图11-12 查询分区 visit_split_date=2022-08-08 的前 5 条数据

从图 11-12 可以看出，中间表 weblog_dws 的分区 visit_split_date=2022-08-08 存在数据，并且在明细表每行数据的基础上，将日期和时间存放在不同的字段中，因此说明成功将明细表 weblog_dws 的处理结果导入到中间表 weblog_dwm。

4. 创建业务表 weblog_pv_dwd

在数据库 weblog_dw 创建业务表 weblog_pv_dwd，命令如下。

```
create table if not exists
weblog_dw.weblog_pv_dwd(
    datestr string,
    pv bigint
)
row format delimited
fields terminated by ','
lines terminated by '\n';
```

5. 创建业务表 weblog_avguv_dwd

在数据库 weblog_dw 创建业务表 weblog_avguv_dwd，命令如下。

```
create table if not exists
weblog_dw.weblog_avguv_dwd(
    datestr string,
    avguv int
)
row format delimited
fields terminated by ','
lines terminated by '\n';
```

11.5 模块开发——数据分析

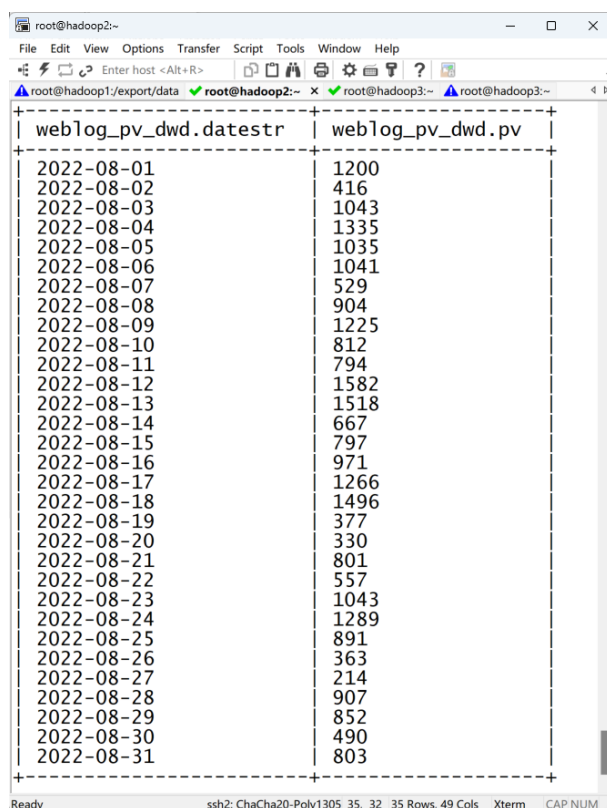
数据仓库构建完成后，用户便可以使用 HiveQL 语句进行数据分析。本项目主要针对浏览量和人均浏览页数这两个需求进行分析，具体内容如下。

1. 浏览量分析

基于中间表 `weblog_dwm` 统计网站的页面每天被用户浏览的次数，并将统计结果插入到业务表 `weblog_pv_dwd`，命令如下。

```
insert into table weblog_dw.weblog_pv_dwd
select visit_split_date,count(*) from weblog_dw.weblog_dwm
group by visit_split_date;
```

上述命令执行完成后，执行“`select * from weblog_dw.weblog_pv_dwd;`”命令，查询业务表 `weblog_pv_dwd` 的数据，如图 11-13 所示。



weblog_pv_dwd.datestr	weblog_pv_dwd.pv
2022-08-01	1200
2022-08-02	416
2022-08-03	1043
2022-08-04	1335
2022-08-05	1035
2022-08-06	1041
2022-08-07	529
2022-08-08	904
2022-08-09	1225
2022-08-10	812
2022-08-11	794
2022-08-12	1582
2022-08-13	1518
2022-08-14	667
2022-08-15	797
2022-08-16	971
2022-08-17	1266
2022-08-18	1496
2022-08-19	377
2022-08-20	330
2022-08-21	801
2022-08-22	557
2022-08-23	1043
2022-08-24	1289
2022-08-25	891
2022-08-26	363
2022-08-27	214
2022-08-28	907
2022-08-29	852
2022-08-30	490
2022-08-31	803

图11-13 查询业务表 `weblog_pv_dwd` 的数据

图 11-13 展示了网站在 2022 年 8 月份每天的浏览量，如 2022 年 08 月 12 日的浏览量为 1582。

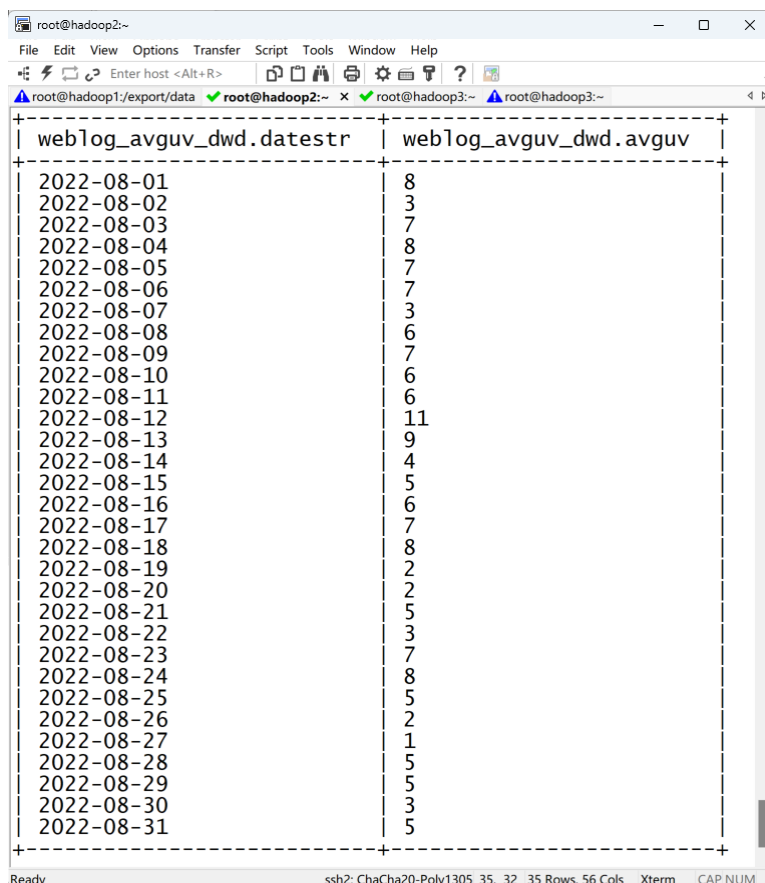
2. 人均浏览页数分析

基于中间表 `weblog_dwm` 统计网站的页面被每个用户平均浏览的次数，并将统计结果插入到业务表 `weblog_avguv_dwd`，命令如下。

```
insert into table weblog_dw.weblog_avguv_dwd
```

```
select visit_split_date,count(*)/count(distinct visit_ip)
from weblog_dw.weblog_dwm group by visit_split_date;
```

上述命令执行完成后，执行“select * from weblog_dw.weblog_avguv_dwd;”命令，查询业务表 weblog_avguv_dwd 的数据，如图 11-14 所示。



weblog_avguv_dwd.datestr	weblog_avguv_dwd.avguv
2022-08-01	8
2022-08-02	3
2022-08-03	7
2022-08-04	8
2022-08-05	7
2022-08-06	7
2022-08-07	3
2022-08-08	6
2022-08-09	7
2022-08-10	6
2022-08-11	6
2022-08-12	11
2022-08-13	9
2022-08-14	4
2022-08-15	5
2022-08-16	6
2022-08-17	7
2022-08-18	8
2022-08-19	2
2022-08-20	2
2022-08-21	5
2022-08-22	3
2022-08-23	7
2022-08-24	8
2022-08-25	5
2022-08-26	2
2022-08-27	1
2022-08-28	5
2022-08-29	5
2022-08-30	3
2022-08-31	5

图11-14 查询业务表 weblog_avguv_dwd 的数据

图 11-14 展示了网站在 2022 年 8 月份每天每个用户平均浏览网站页面的次数，如 2022 年 08 月 12 日每个用户平均浏览网站页面的次数为 11。

需要注意的是，读者在查询业务表 weblog_pv_dwd 和 weblog_avguv_dwd 的数据时，其查询结果与图 11-13 和图 11-14 展示的查询结果会有所不同，这是因为本项目是模拟生成的日志数据，导致每位读者使用 Flume 采集日志数据的内容也会有所不同。除此之外，Flume 采集数据量的大小也会对最终分析结果产生影响。

11.6 模块开发——数据导出

为了方便后续进行数据可视化，使用 Sqoop 将 Hive 数据仓库中业务表 weblog_pv_dwd 和 weblog_avguv_dwd 的数据导出到 MySQL 的数据表，具体操作步骤如下。

(1) 在虚拟机 Hadoop3 登录 MySQL，命令如下。

```
$ mysql -uroot -pItcast@2022
```

(2) 在 MySQL 创建数据库 `sqoop_weblog`，该数据库用于存放本项目相关的数据表，命令如下。

```
mysql> CREATE DATABASE `sqoop_weblog` DEFAULT CHARACTER SET utf8 ;
```

(3) 在数据库 `sqoop_weblog` 中创建数据表 `weblog_pv_dwd`，该数据表用于存放 Hive 数据仓库中业务表 `weblog_pv_dwd` 导出的数据，命令如下。

```
mysql> CREATE TABLE `sqoop_weblog`.`weblog_pv_dwd` (  
    `datestr` VARCHAR(20),  
    `pv` INT)  
ENGINE = InnoDB  
DEFAULT CHARACTER SET = utf8;
```

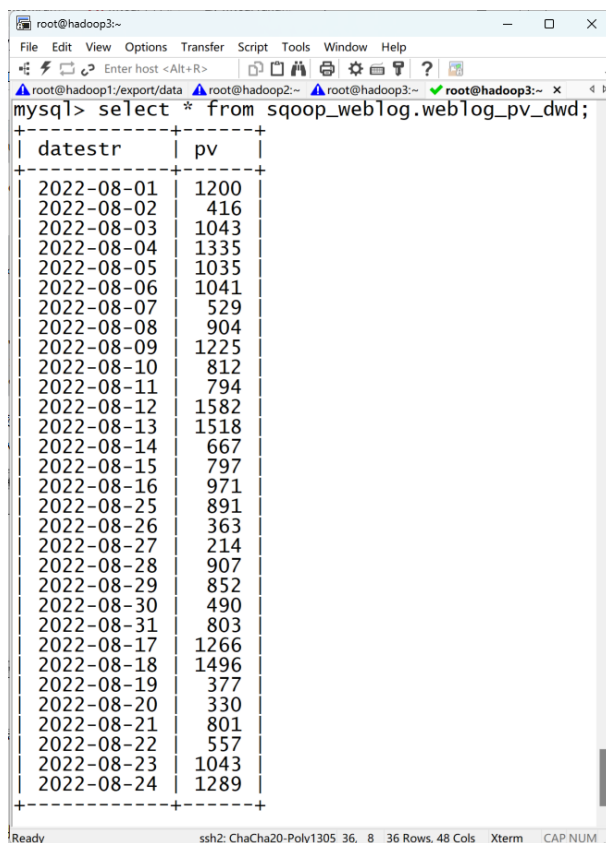
(4) 在数据库 `sqoop_weblog` 中创建数据表 `weblog_avguv_dwd`，该数据表用于存放 Hive 数据仓库中业务表 `weblog_avguv_dwd` 导出的数据，命令如下。

```
mysql> CREATE TABLE `sqoop_weblog`.`weblog_avguv_dwd` (  
    `datestr` VARCHAR(20),  
    `avguv` INT)  
ENGINE = InnoDB  
DEFAULT CHARACTER SET = utf8;
```

(5) 通过 Sqoop 提供的 `export` 命令，将 Hive 数据仓库中业务表 `weblog_pv_dwd` 的数据导出到 MySQL 的数据表 `weblog_pv_dwd`，命令如下

```
sqoop export \  
--connect jdbc:mysql://hadoop3:3306/sqoop_weblog?allowPublicKeyRetrieval=true \  
--username root \  
--password Itcast@2022 \  
--columns datestr,pv \  
--table weblog_pv_dwd \  
--export-dir /user/hive_local/warehouse/weblog_dw.db/weblog_pv_dwd \  
--num-mappers 1
```

执行完上述命令后，在 MySQL 中执行 “`select * from sqoop_weblog.weblog_pv_dwd;`” 命令，查询数据表 `weblog_pv_dwd` 的数据，如图 11-15 所示。



datestr	pv
2022-08-01	1200
2022-08-02	416
2022-08-03	1043
2022-08-04	1335
2022-08-05	1035
2022-08-06	1041
2022-08-07	529
2022-08-08	904
2022-08-09	1225
2022-08-10	812
2022-08-11	794
2022-08-12	1582
2022-08-13	1518
2022-08-14	667
2022-08-15	797
2022-08-16	971
2022-08-25	891
2022-08-26	363
2022-08-27	214
2022-08-28	907
2022-08-29	852
2022-08-30	490
2022-08-31	803
2022-08-17	1266
2022-08-18	1496
2022-08-19	377
2022-08-20	330
2022-08-21	801
2022-08-22	557
2022-08-23	1043
2022-08-24	1289

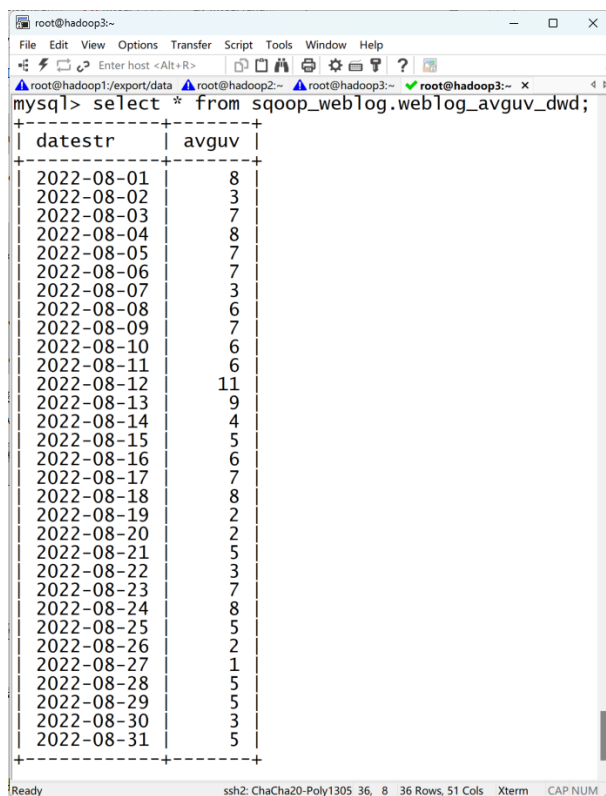
图11-15 查询数据表 weblog_pv_dwd 的数据

从图 11-15 可以看出，数据表 weblog_pv_dwd 的数据与 Hive 数据仓库中业务表 weblog_pv_dwd 的数据一致，因此说明成功实现了数据导出。

(6) 通过 Sqoop 提供的 export 命令，将 Hive 数据仓库中业务表 weblog_avguv_dwd 的数据导出到 MySQL 的数据表 weblog_avguv_dwd，命令如下。

```
sqoop export \
--connect jdbc:mysql://hadoop3:3306/sqoop_weblog?allowPublicKeyRetrieval=true \
--username root \
--password Itcast@2022 \
--columns datestr,avguv \
--table weblog_avguv_dwd \
--export-dir /user/hive_local/warehouse/weblog_dw.db/weblog_avguv_dwd \
--num-mappers 1
```

执行完上述命令后，在 MySQL 中执行“select * from sqoop_weblog.weblog_avguv_dwd;”命令，查询数据表 weblog_pv_dwd 的数据，如图 11-16 所示。



datestr	avguv
2022-08-01	8
2022-08-02	3
2022-08-03	7
2022-08-04	8
2022-08-05	7
2022-08-06	7
2022-08-07	3
2022-08-08	6
2022-08-09	7
2022-08-10	6
2022-08-11	6
2022-08-12	11
2022-08-13	9
2022-08-14	4
2022-08-15	5
2022-08-16	6
2022-08-17	7
2022-08-18	8
2022-08-19	2
2022-08-20	2
2022-08-21	5
2022-08-22	3
2022-08-23	7
2022-08-24	8
2022-08-25	5
2022-08-26	2
2022-08-27	1
2022-08-28	5
2022-08-29	5
2022-08-30	3
2022-08-31	5

图11-16 查询数据表 weblog_avguv_dwd 的数据

从图 11-16 可以看出，数据表 weblog_avguv_dwd 的数据与 Hive 数据仓库中业务表 weblog_avguv_dwd 的数据一致，因此说明成功实现了数据导出。

11.7 模块开发——数据可视化

数据可视化是指将数据或信息表示为图形中的可视对象来传达数据或信息的技术，目的是清晰有效地向用户传达信息，以便用户可以轻松了解数据或信息中的复杂关系。用户可以通过图形中的可视对象直观地看到数据分析结果，从而更容易理解业务变化趋势或发现新的业务模式。数据可视化是数据分析中的一个重要步骤。本节将讲解如何使用 FineBI 工具实现数据可视化。

11.7.1 安装、启动与配置 FineBI

FineBI 是帆软软件有限公司推出的一款商业智能（BI，Business Intelligence）分析工具，它提供一套完整的解决方案，用来将企业中现有的数据进行有效的整合，快速准确的提供报表并提出决策依据，帮助企业做出明智的业务经营决策。

本项目使用 FineBI 的数据可视化功能，将浏览量分析和人均浏览页数分析的结果进行可视化展示。接下来，将详细讲解如何安装、启动和配置 FineBI，具体步骤如下。

1. 下载 FineBI

读者可自行访问 FineBI 官网，通过注册免费试用的方式下载 Windows x64 位操作系统的 FineBI 安装包 windows-x64_FineBI5_1-CN.exe。

2. 安装 FineBI

双击 FineBI 安装包 windows-x64_FineBI5_1-CN.exe，进入 FineBI 安装向导界面，如图 11-17 所示。

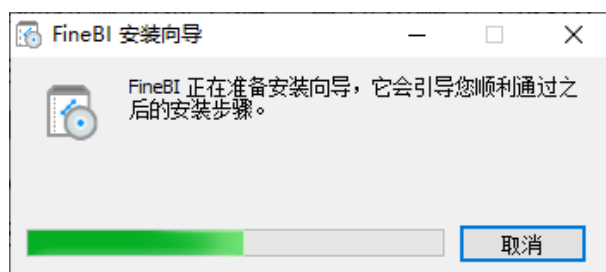


图11-17 FineBI 安装向导界面

在图 11-17 所示界面等待准备工作完成之后，进入欢迎使用 FineBI 安装程序向导界面，如图 11-18 所示。



图11-18 欢迎使用 FineBI 安装程序向导界面

在图 11-18 中，单击“下一步”按钮进入许可协议界面，在该界面勾选“我接受协议”选项，如图 11-19 所示。

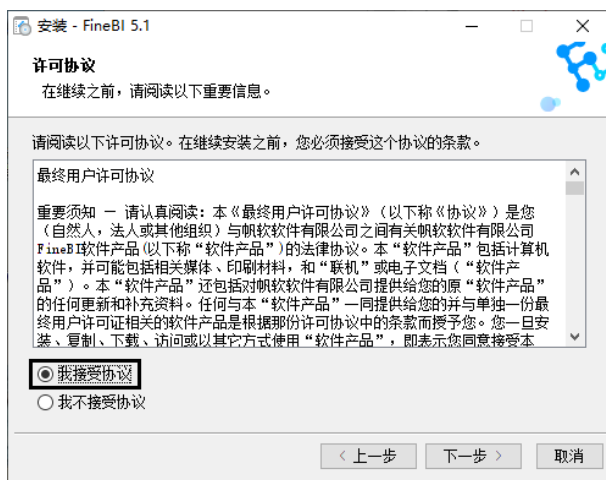


图11-19 许可协议界面

在图 11-19 中，单击“下一步”按钮进入选择安装目录界面，在该界面中配置 FineBI 的安装目录，这里配置 FineBI 的安装目录位 D:\FineBI5.1，如图 11-20 所示。

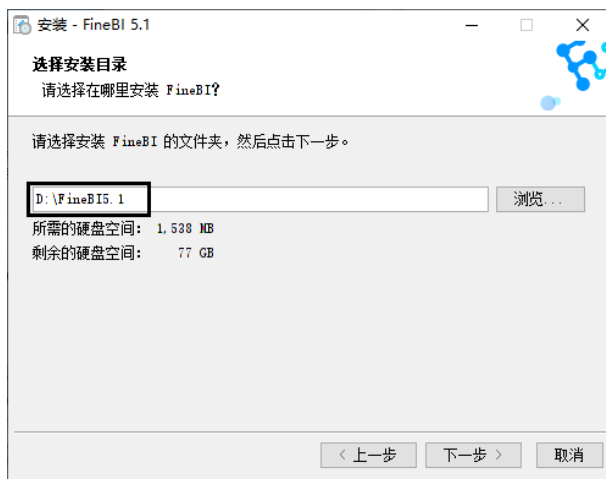


图11-20 选择安装目录界面

在图 11-20 中，单击“下一步”按钮进入设置最大内存界面，在“最大 jvm 内存”输入框中输入最大 jvm 内存，读者可以根据实际情况去设置，但建议设置最大 jvm 内存至少为 2048（2GB），这里设置最大 jvm 内存位 4096（4GB），如图 11-21 所示。

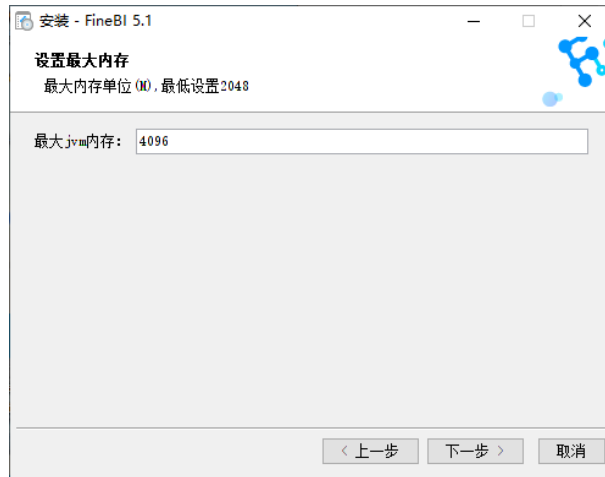


图11-21 设置最大内存界面

在图 11-21 中，单击“下一步”按钮进入选择开始菜单文件夹界面，在该界面使用默认配置即可，如图 11-22 所示。

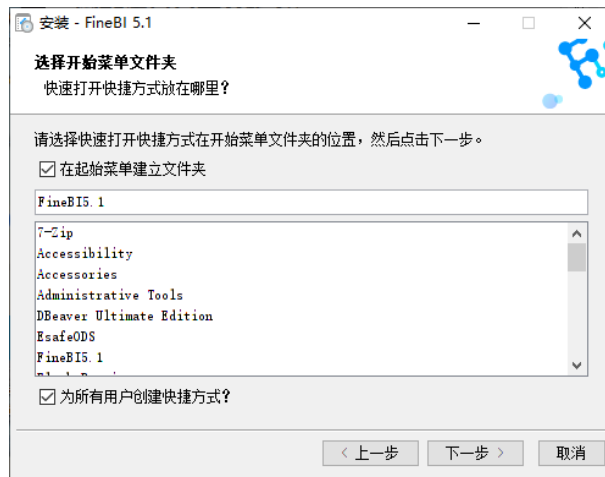


图11-22 选择开始菜单文件夹界面

在图 11-22 中，单击“下一步”按钮进入选择附加工作界面，在该界面使用默认配置即可，如图 11-23 所示。

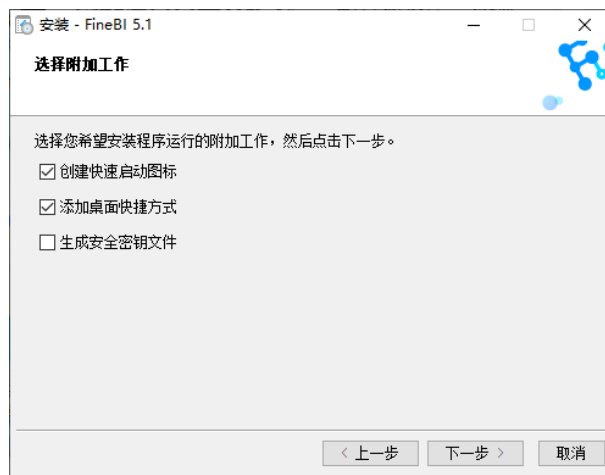


图11-23 选择附加工作界面

在图 11-23 中，单击“下一步”按钮进入安装中界面，在该界面 FineBI 会自动安装，如图 11-24 所示。

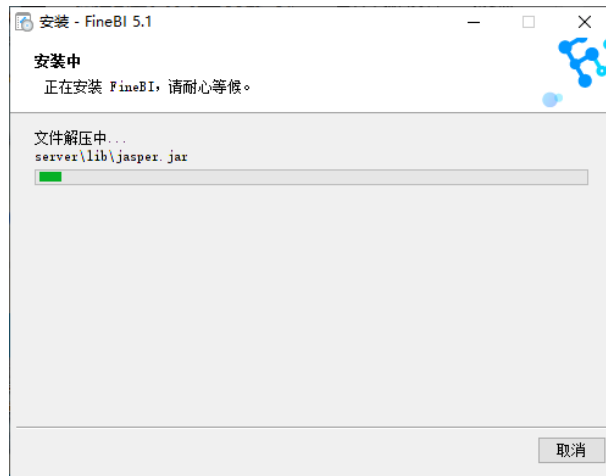


图11-24 安装中界面

在图 11-24 中，等待 FineBI 安装完成后会进入完成 FineBI 安装程序界面，在该界面使用默认配置即可，如图 11-25 所示。



图11-25 完成 FineBI 安装程序界面

在图 11-25 中，单击“完成”按钮，由于在图 11-25 中默认配置勾选了“运行 FineBI”选项，所以此时会自动运行 FineBI，FineBI 启动界面如图 11-26 所示。



图11-26 FineBI 启动界面

等待 FineBI 启动完成后，系统默认配置的浏览器会打开 FineBI 平台，如图 11-27 所示。

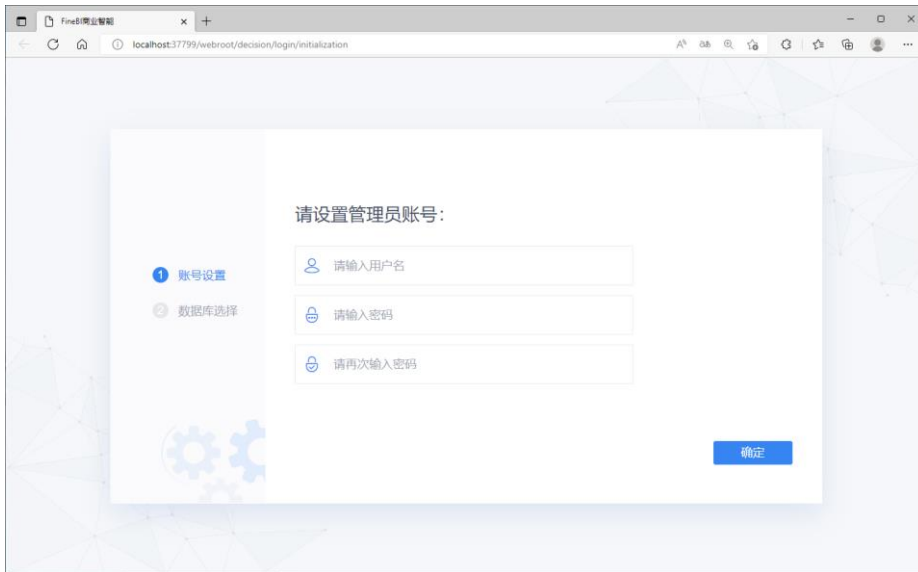


图11-27 FineBI 平台

从图 11-27 可以看出，FineBI 平台默认的 URL 地址为 <http://localhost:37799/webroot/decision/login/initialization>。

至此，便完成了 FineBI 的安装与启动，后续可以通过桌面生成的 FineBI 快捷图标启动 FineBI。需要注意的是，如果 FineBI 安装完成后无法进入 FineBI 平台，在弹出的页面提示“内置数据库连接失败”的信息，那么读者可以关闭当前启动的 FineBI，并以系统管理员的身份重新启动 FineBI。

3. 配置 FineBI 登录用户

首次使用 FineBI 时需要通过 FineBI 平台进行初始化设置，在图 11-27 中设置管理员账号，这里将用户名设置为 itcast，密码设置为 123456，如图 11-28 所示。

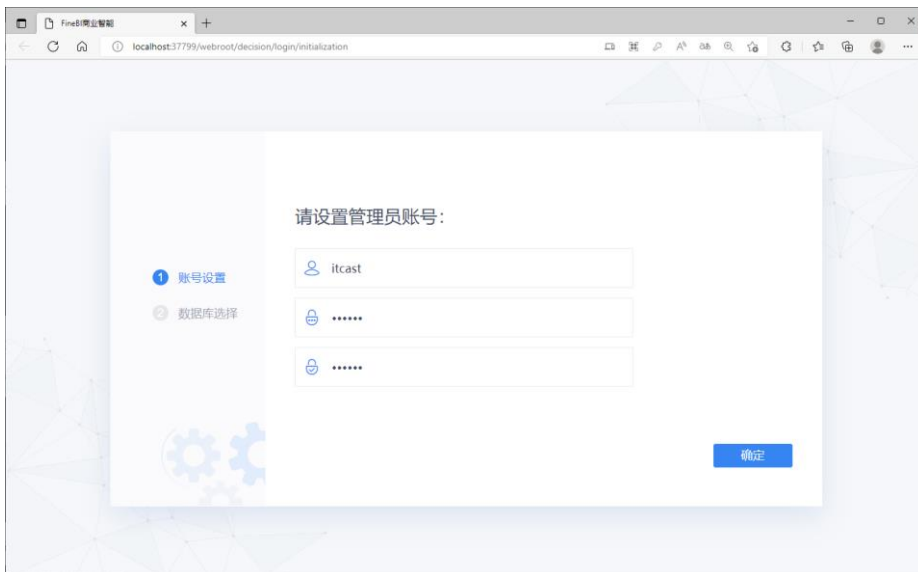


图11-28 设置管理员账号

在图 11-28 中，单击“确定”按钮，FineBI 平台会显示“管理员账号设置成功”信息，并且设置的管理员密码会以明文的方式显示，如图 11-29 所示。

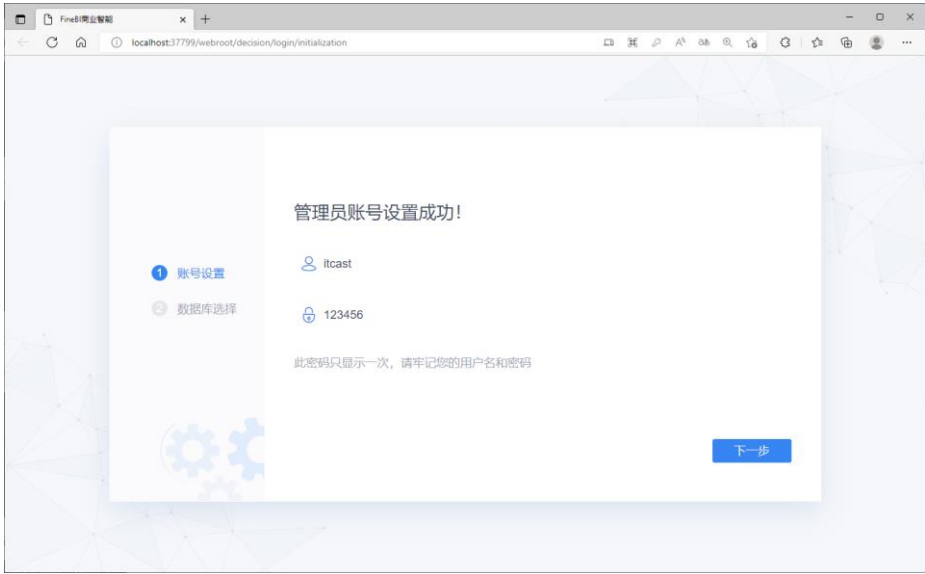


图11-29 管理员账号设置成功

在图 11-29 中，单击“下一步”按钮选择 FineBI 使用的数据库，如图 11-30 所示。

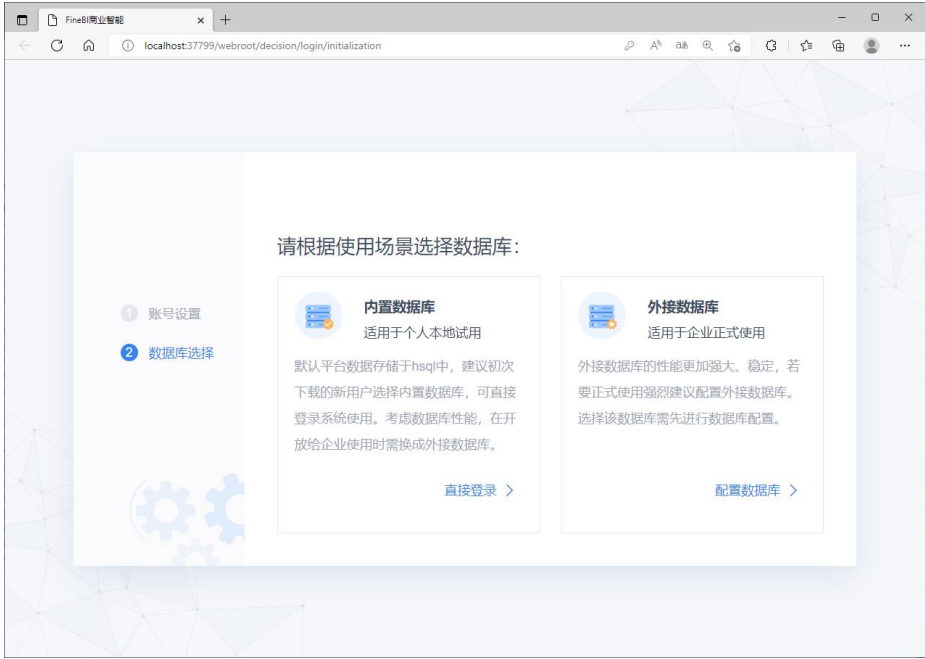


图11-30 选择 FineBI 使用的数据库

在图 11-30 中，单击“直接登录”按钮，页面会自动跳转到 FineBI 的登录界面，如图 11-31 所示。

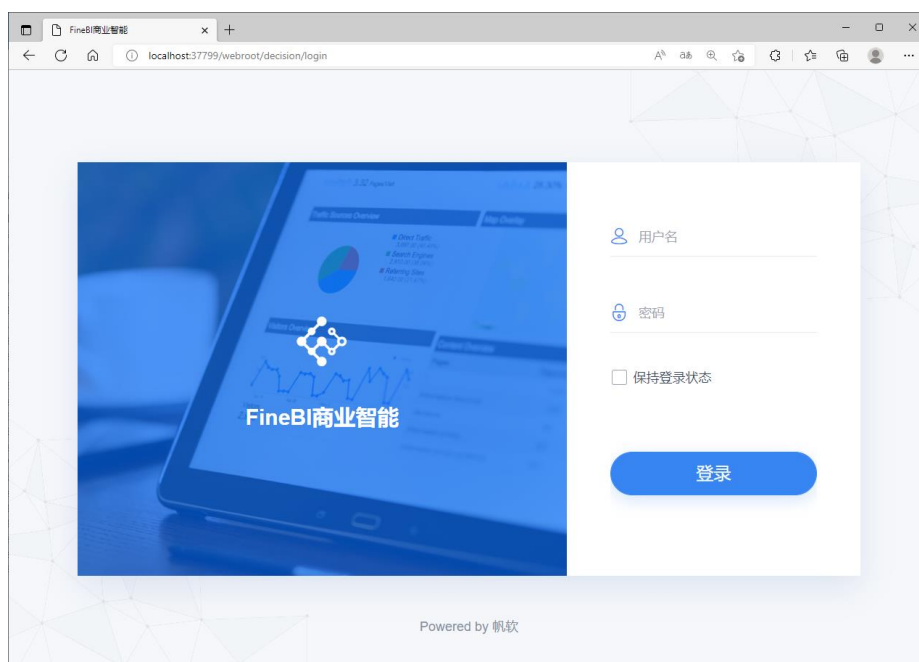


图11-31 FineBI 登录界面

在图 11-31 中，输入图 11-29 中设置的用户名 itcast 和密码 123456，单击“登录”按钮登录 FineBI，如图 11-32 所示。



图11-32 登录 FineBI

至此，便完成了配置 FineBI 登录用户。

4. 配置 FineBI 数据连接

数据连接用于配置 FineBI 使用的数据库，本项目通过 FineBI 获取虚拟机 Hadoop3 中 MySQL 的数据进行可视化展示，有关 FineBI 连接 MySQL 的操作步骤如下。

(1) 依次单击 FineBI 的“管理系统”→“数据连接”→“数据连接管理”按钮进入数据连接管理界面，如图 11-33 所示。

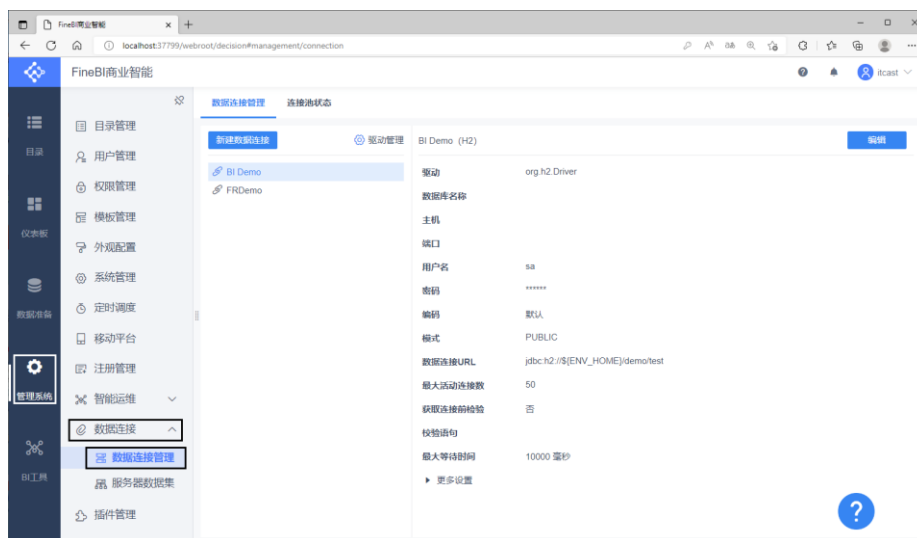


图11-33 数据连接管理界面

(2) 在图 11-33 中，单击“新建数据连接”按钮选择 FineBI 使用的数据库，如图 11-34 所示。

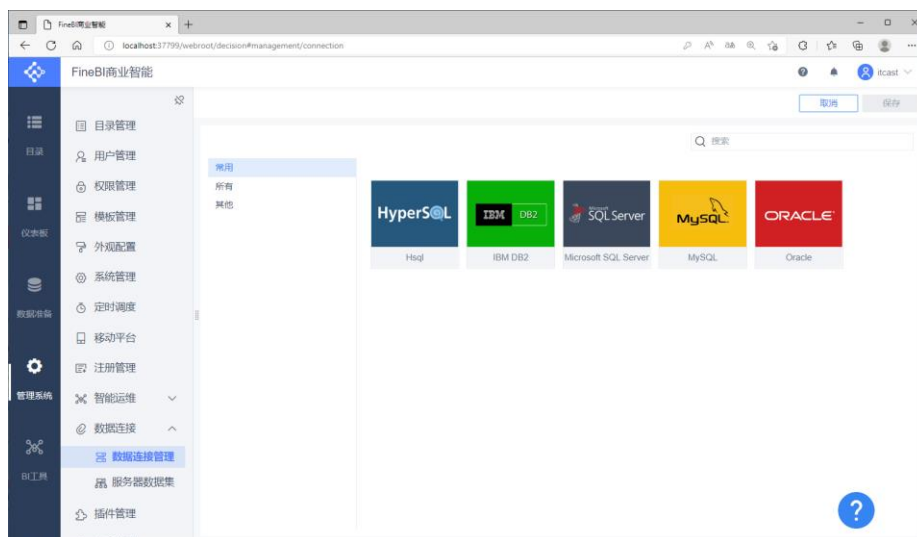


图11-34 选择 FineBI 使用的数据库

(3) 在图 11-34 展示的数据库列表中选择“MySQL”选项，配置连接 MySQL 的相关信息，完成 MySQL 相关信息配置的效果如图 11-35 所示。

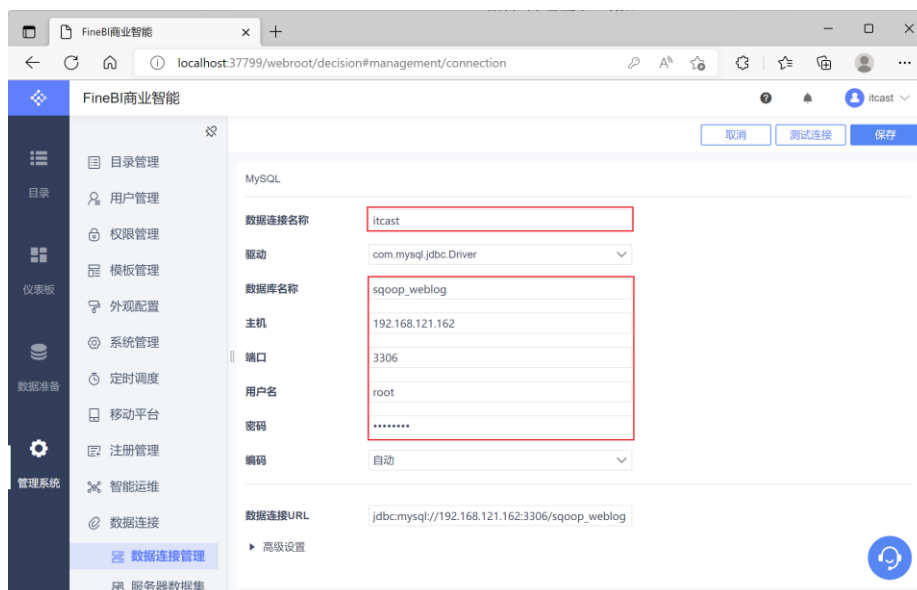


图11-35 配置 MySQL 相关信息

在图 11-35 中，修改数据连接名称为 itcast；修改数据库名称为 sqoop_weblog；填写端口为 3306，即 MySQL 数据库默认端口；填写主机为 192.168.121.162，即虚拟机 Hadoop3 的 IP 地址；填写用户名“root”；填写密码“Itcast@2022”。

（4）在图 11-35 中，单击“测试连接”按钮，验证 FineBI 是否可以成功连接虚拟机 Hadoop3 的 MySQL，如图 11-36 所示。

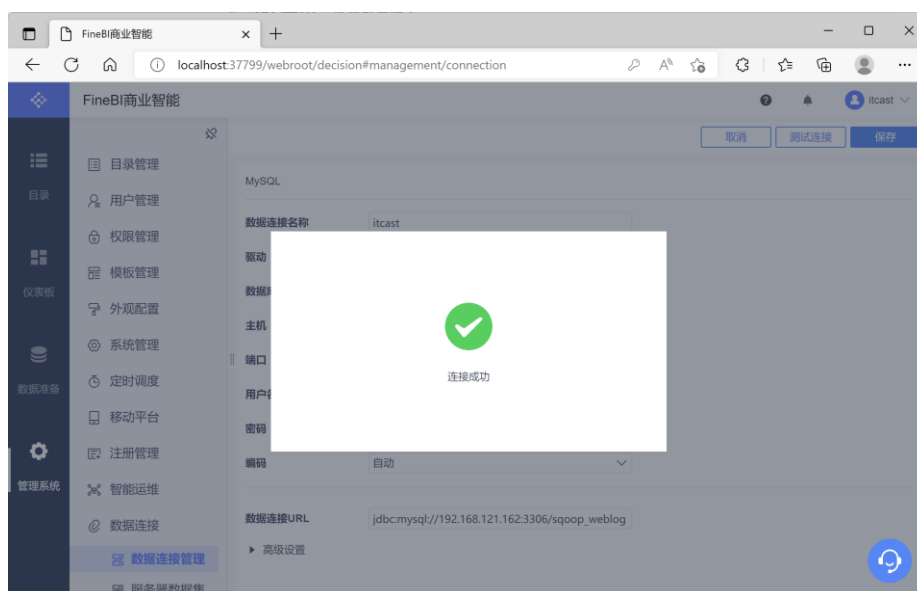


图11-36 测试连接

从图 11-36 中可以看出，弹出“连接成功”提示信息，说明 FineBI 成功连接虚拟机 Hadoop3 的 MySQL，此时单击图 11-35 中的“保存”按钮即可。

11.7.2 实现数据可视化

通过 11.7.1 小节的操作，我们已经成功完成了 FineBI 的安装、启动和配置，接下来，我们讲解如何通过 FineBI 实现数据可视化。

1. 配置 FineBI 数据准备

数据准备用于配置 FineBI 使用的数据内容，主要是选择数据库中要使用的表，有关 FineBI 数据准备的操作步骤如下。

(1) 选择 FineBI “数据准备” 选项，如图 11-37 所示。

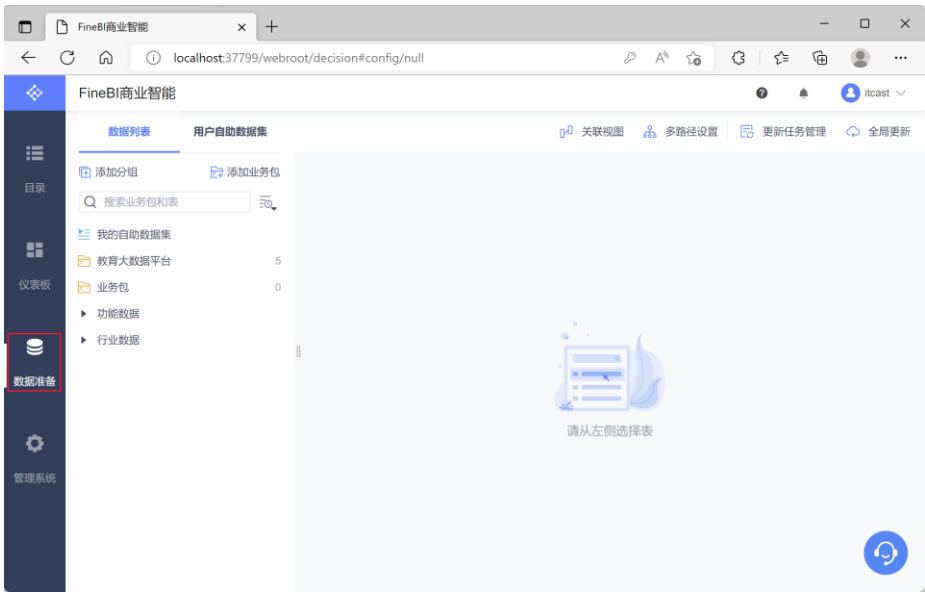


图11-37 数据准备

(2) 在图 11-37 中，单击“添加业务包”按钮添加业务，并配置业务名称为网站流量日志数据分析系统，如图 11-38 所示。

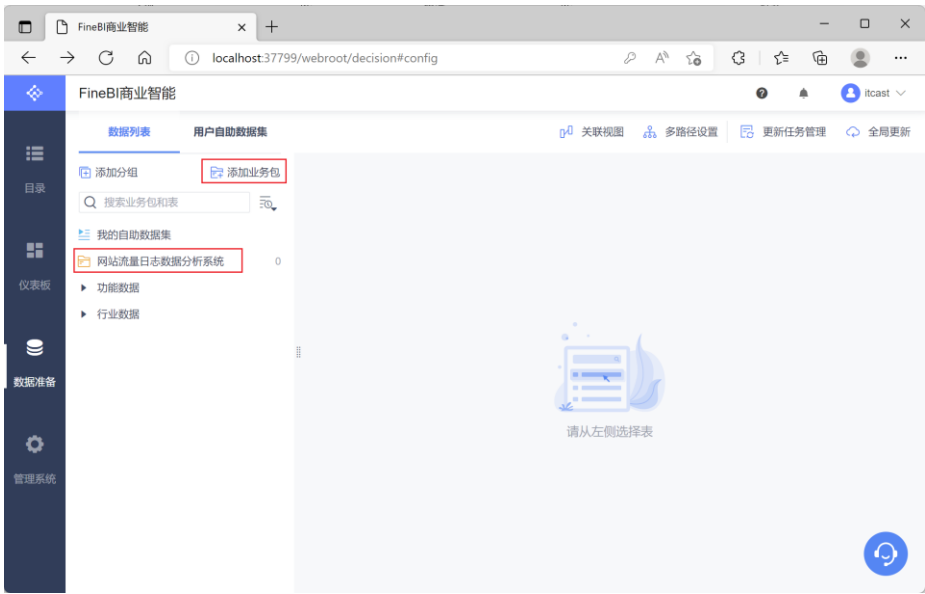


图11-38 配置业务名称

(3) 在图 11-38 中,单击“网站流量日志数据分析系统”配置该业务使用的数据内容,如图 11-39 所示。

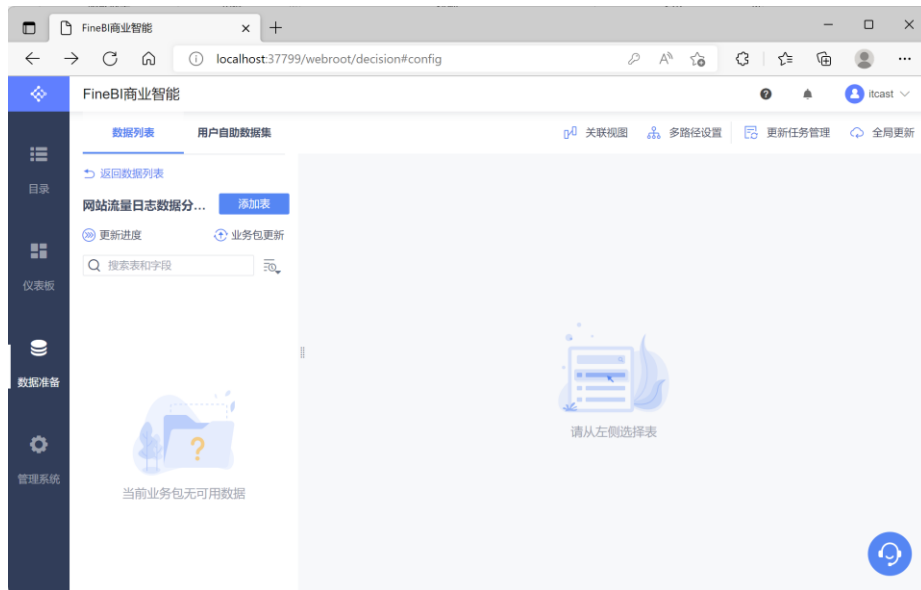


图11-39 配置业务使用的数据内容

(4) 在图 11-39 中,依次单击“添加表”→“数据库表”按钮选择需要使用的数据表,这里选择数据表 weblog_avguv_dwd 和 weblog_pv_dwd,如图 11-40 所示。

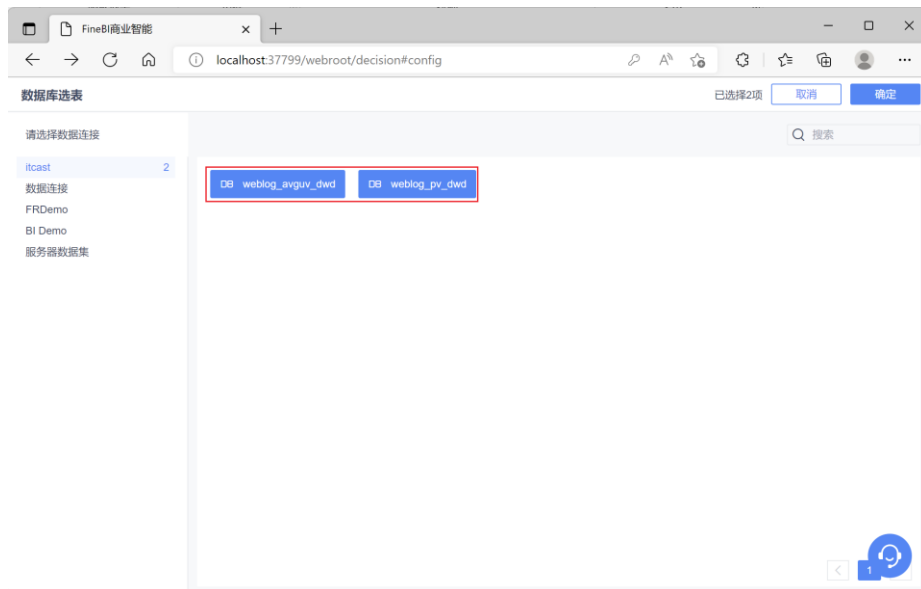


图11-40 选择需要使用的数据表

(5) 在图 11-40 中,单击“确定”按钮添加数据表 weblog_avguv_dwd 和 weblog_pv_dwd,此时页面会跳转到图 11-39 所示的页面,如图 11-41 所示。

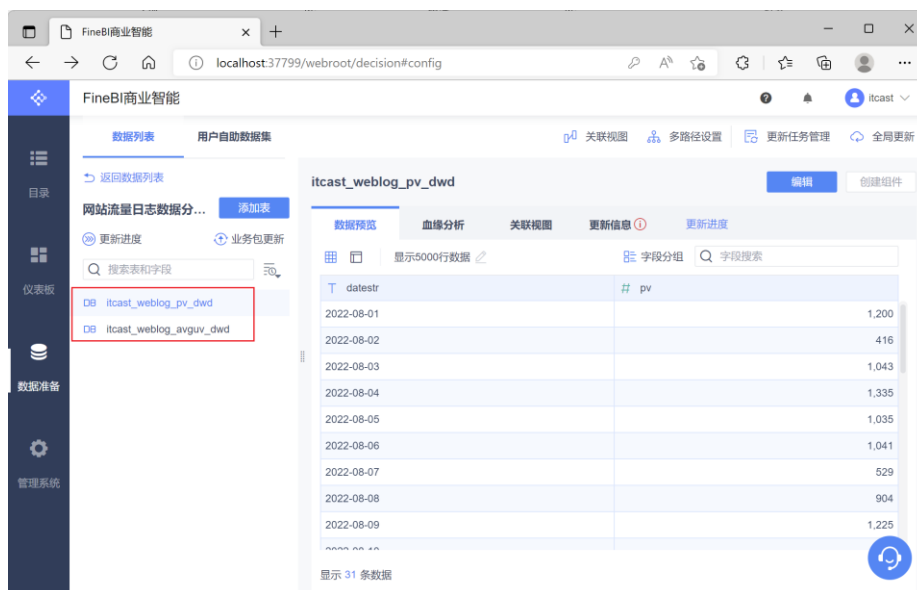


图11-41 添加数据表 weblog_avguv_dwd 和 weblog_pv_dwd

(6) 在图 11-41 中，单击“业务包更新”按钮，更新业务包“网站流量日志数据分析系统”中添加的数据表 weblog_avguv_dwd 和 weblog_pv_dwd，否则后续无法使用这两个表的数据进行可视化展示，如图 11-42 所示。

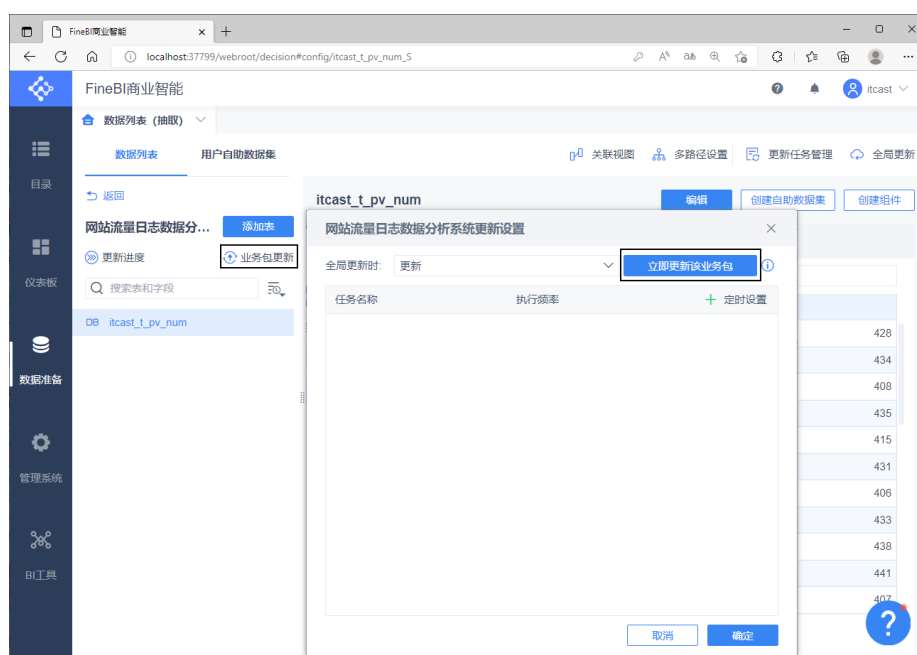


图11-42 业务包更新

在图 11-42 中，单击“立即更新该业务包”按钮，等待更新完成后，单击“确定”按钮即可。

2. 配置 FineBI 仪表板

仪表板用于 FineBI 进行数据可视化展示的画板，有关配置 FineBI 仪表板的操作步骤如下。

(1) 单击 FineBI 的“仪表版”选项进入仪表板管理页面，在该页面单击“新建仪表板”按钮，在弹出窗口的“名称”文本框中输入“网站流量日志数据分析系统”，如图 11-43 所示。

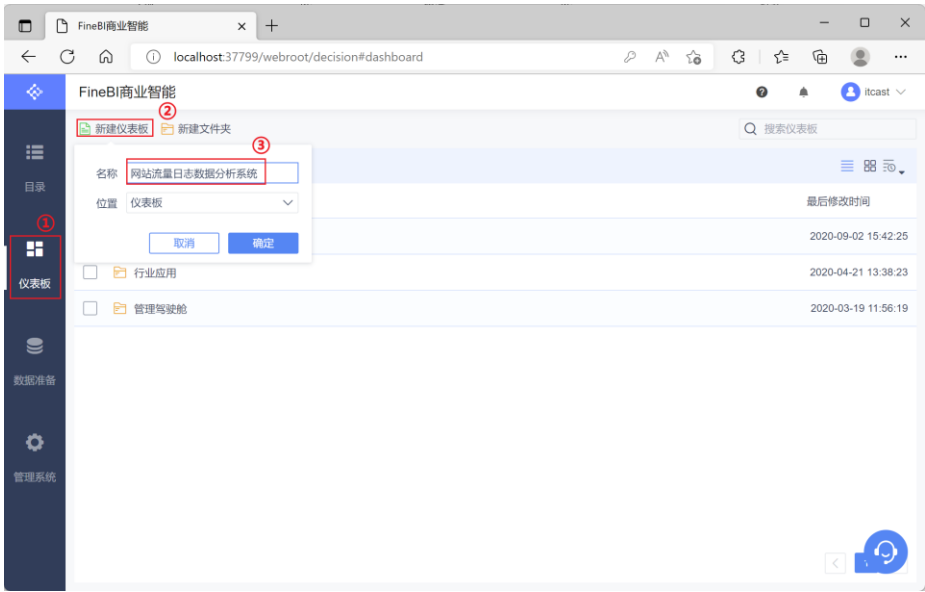


图11-43 新建仪表板

(2) 在图 11-43 中，单击“确认”按钮创建仪表板网站流量日志数据分析系统，此时会跳转到该仪表板的配置界面，如图 11-44 所示。

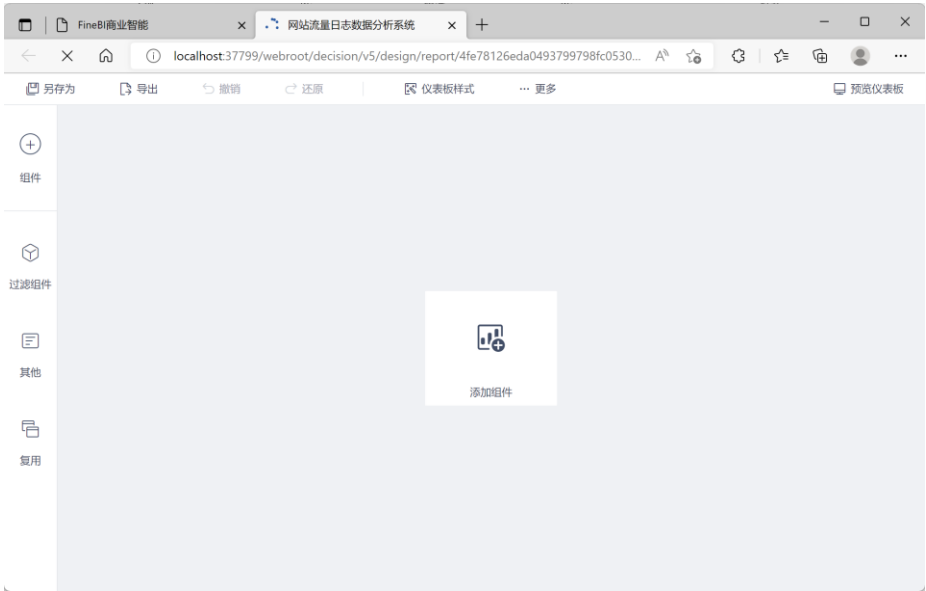


图11-44 仪表板的配置页面

此时，已经完成了配置 FineBI 仪表板的操作。

3. 实现浏览量分析结果的可视化

通过 FineBI 提供的折线图，实现浏览量分析结果的可视化，其操作步骤如下。

(1) 在图 11-44 中，单击“添加组件”按钮，进入到添加组件页面，如图 11-45 所示。

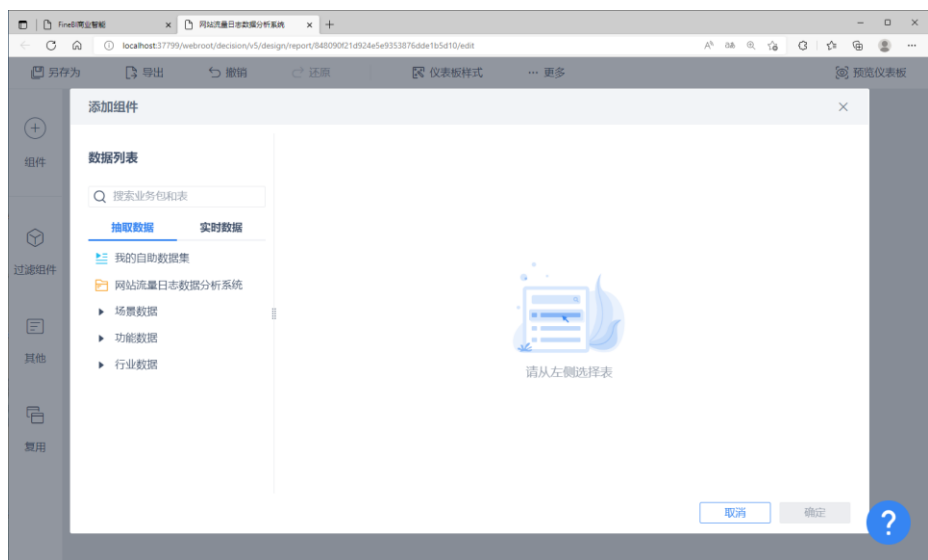


图11-45 添加组件页面

(2) 在图 11-45 中，单击“网站流量日志数据分析系统”业务包，然后在弹出的页面选择数据库 itcast 中的数据表 weblog_pv_dwd，如图 11-46 所示。

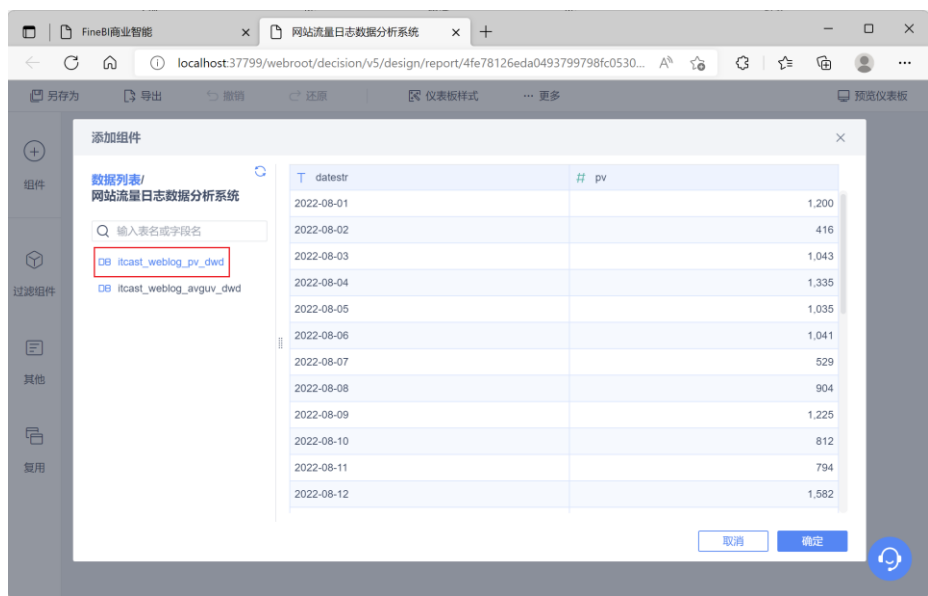


图11-46 选择数据库 itcast 中的数据表 weblog_pv_dwd

(3) 在图 11-46 中，单击“确定”按钮，进入组件的配置页面，如图 11-47 所示。

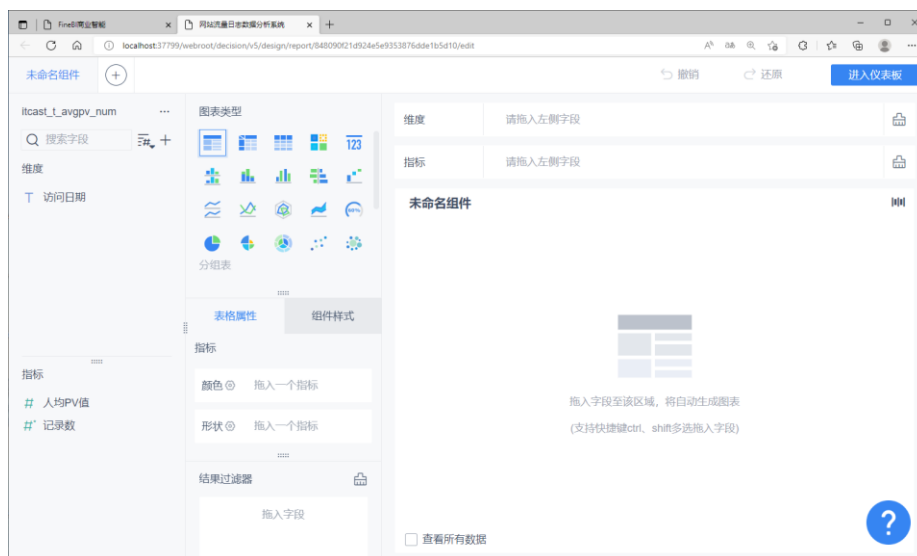


图11-47 组件的配置页面（1）

（4）在图 11-47 中，单击“未命名组件”选项，在弹出的编辑标题页面，指定组件的名称为浏览量分析，如图 11-48 所示。

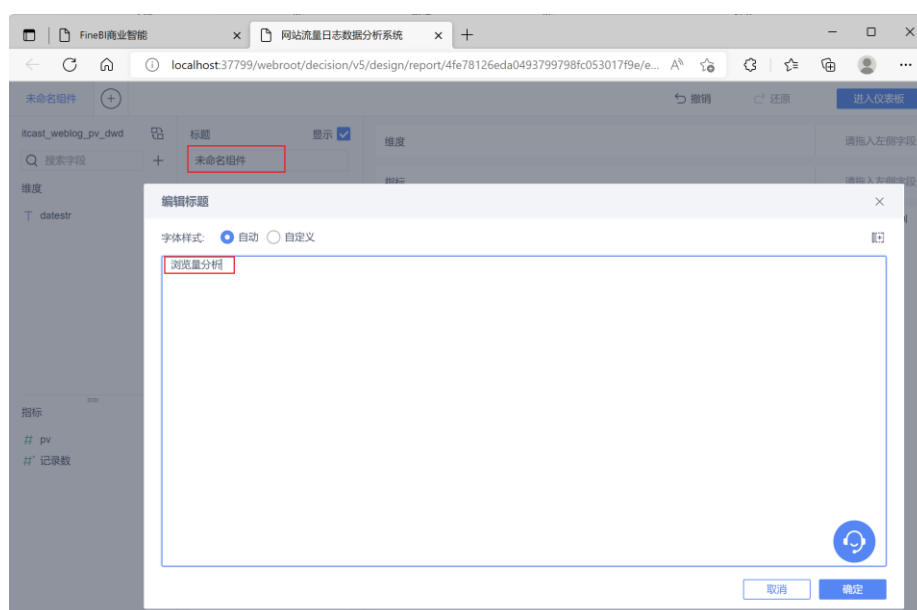


图11-48 命名组件名称（1）

（5）在图 11-48 中，单击“确定”按钮，返回至图 11-47 所示页面，在该页面单击“分区折线图”图标使用分区折线图，如图 11-49 所示。

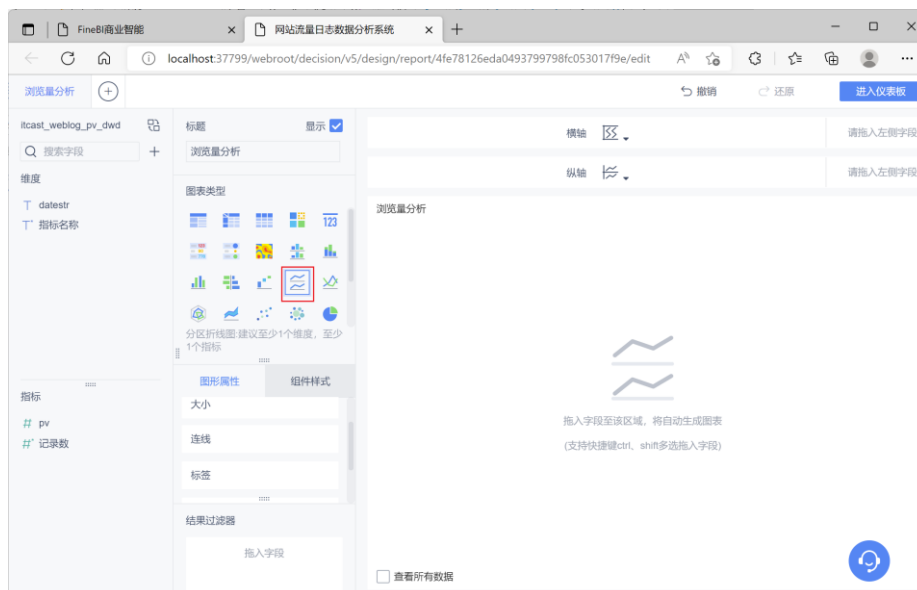


图11-49 使用分区折线图

(6) 在图 11-49 中，首先将维度一栏中的“datestr”拖入“横轴”，然后将指标一栏中的“pv”拖入“纵轴”和图形属性中的“标签”，如图 11-50 所示。

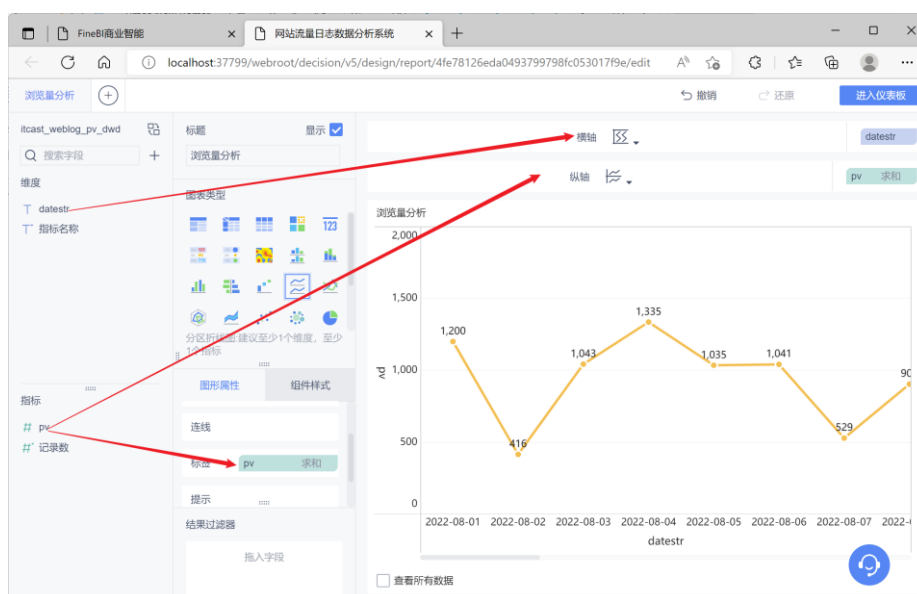


图11-50 配置分区折线图

(7) 在图 11-50 中，单击“进入仪表盘”按钮，进入仪表盘“网站流量日志数据分析系统”，如图 11-51 所示。

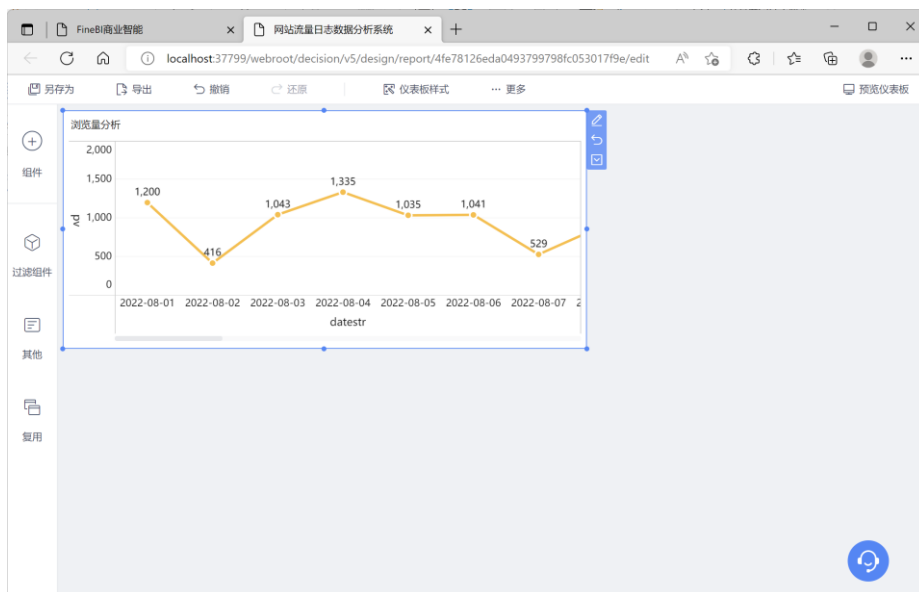


图11-51 实现浏览量分析结果可视化

至此，便成功在“网站流量日志数据分析系统”仪表板中添加了浏览量分析结果的可视化展示组件。

4. 实现人均浏览页数分析结果的可视化

通过 FineBI 提供的柱状图，在“网站流量日志数据分析系统”仪表板中实现人均浏览页数分析结果的可视化，其操作步骤如下。

(1) 在图 11-51 中，单击“组件”按钮，然后在弹出的页面选择数据库 itcast 中的数据表 weblog_avguv_dwd，如图 11-52 所示。

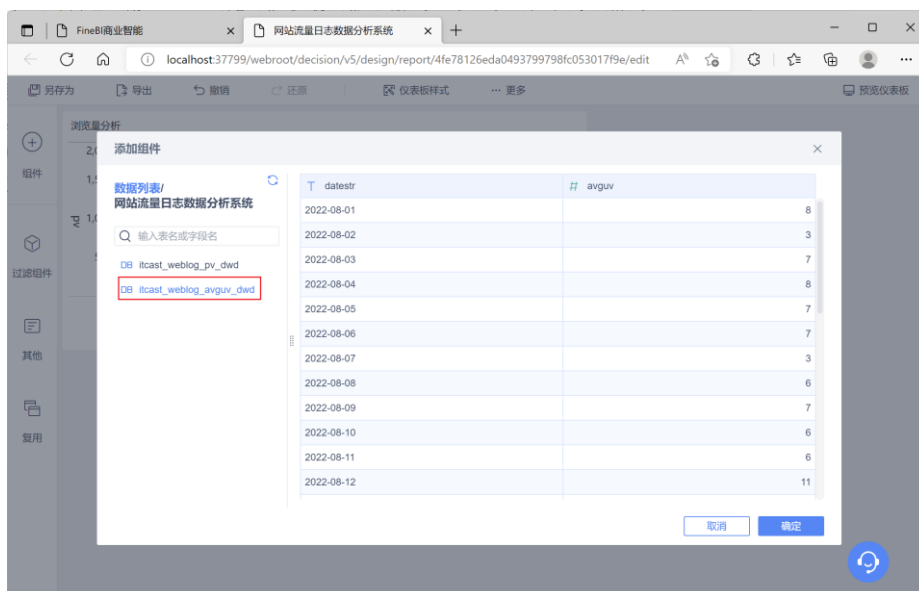


图11-52 选择数据库 itcast 中的数据表 weblog_avguv_dwd

(2) 在图 11-52 中，单击“确定”按钮，进入组件的配置页面，如图 11-53 所示。

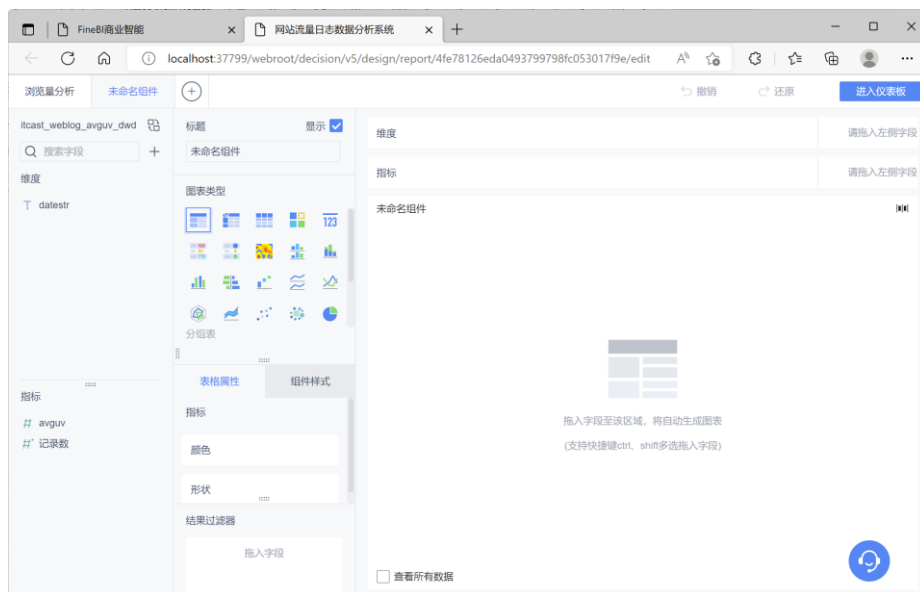


图11-53 组件的配置页面（2）

（3）在图 11-53 中，单击“未命名组件”选项，在弹出的编辑标题页面，指定组件的名称为“人均浏览页数分析”，如图 11-54 所示。

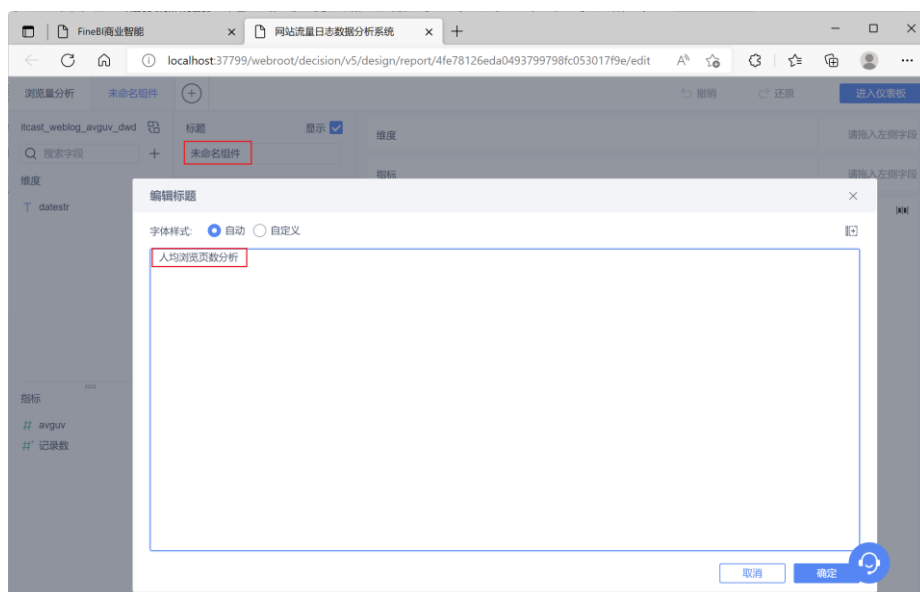


图11-54 命名组件名称（2）

（4）在图 11-54 中，单击“确定”按钮，返回至图 11-53 所示页面，在该页面单击“多系列柱状图”图标使用多系列柱状图，如图 11-55 所示。

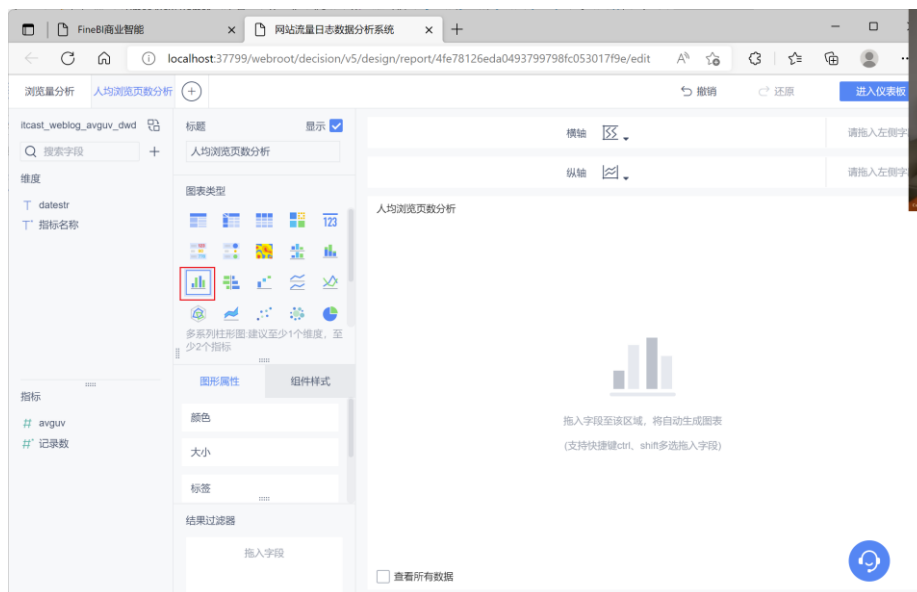


图11-55 使用多系列柱状图

(8) 在图 11-55 中，首先将维度一栏中的“datestr”拖入“横轴”，然后将指标一栏中的“avguv”拖入“纵轴”和图形属性中的“标签”，如图 11-56 所示。

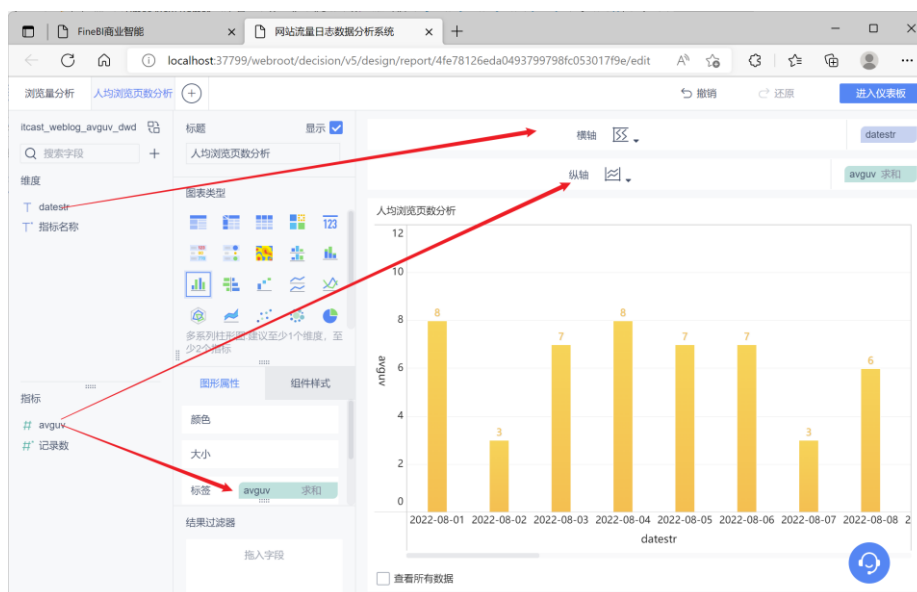


图11-56 配置多系列柱状图

(9) 在图 11-56 中，单击“进入仪表盘”按钮，进入仪表盘“网站流量日志数据分析系统”，如图 11-57 所示。

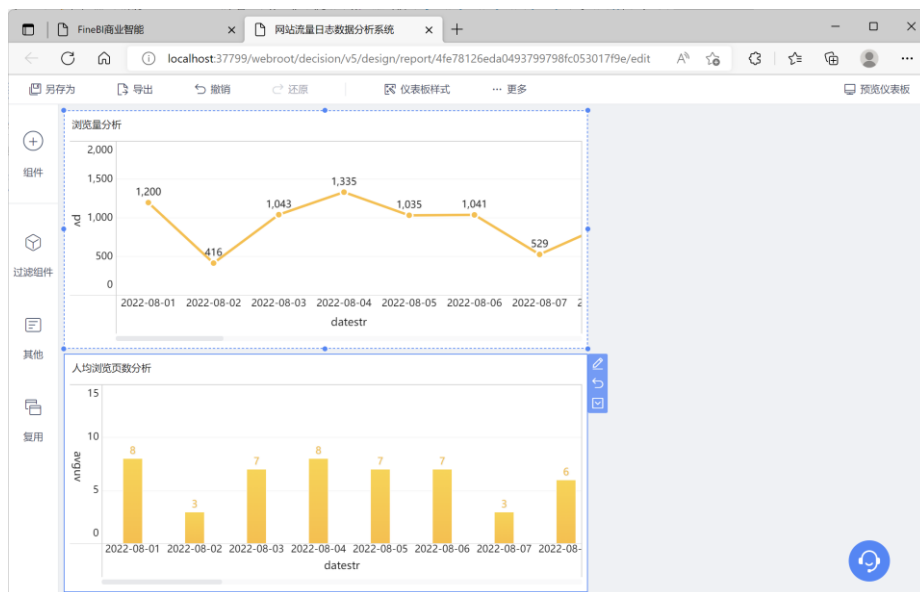


图11-57 实现人均访问页数分析结果可视化

至此，便成功在“网站流量日志数据分析系统”仪表板中添加了人均访问页数分析结果的可视化展示组件。